



哈爾濱工業大學 (深圳)  
HARBIN INSTITUTE OF TECHNOLOGY

## 实验报告

开课学期: 2022 夏季

课程名称: 计算机设计与实践

实验名称: CPU 设计

实验性质: 综合设计型

实验学时: 52 地点: T2210

学生班级: 14 班

学生学号: 200111407

学生姓名: 刘玄昊

评阅教师: \_\_\_\_\_

报告成绩: \_\_\_\_\_

实验与创新实践教育中心制

2022 年 7 月

注：本设计报告中各个部分如果页数不够，请同学们自行扩页。原则上一定要把报告写详细，能说明设计的成果、特色和过程。报告应该详细叙述整体设计，以及设计中的每个模块。设计报告将是评定每个人成绩的重要组成部分（**设计内容及报告写作**都作为评分依据）。

设计的功能描述（含所有实现的指令描述，以及单周期/流水线 CPU 频率）

实现指令

单周期与流水线 CPU 实现指令相同，为 24 条基础指令

指令描述

单周期

#### R 型

|     |                                         |
|-----|-----------------------------------------|
| add | $(rd) \leftarrow (rs1) + (rs2)$         |
| sub | $(rd) \leftarrow (rs1) - (rs2)$         |
| and | $(rd) \leftarrow (rs1) \& (rs2)$        |
| or  | $(rd) \leftarrow (rs1)   (rs2)$         |
| xor | $(rd) \leftarrow (rs1) \wedge (rs2)$    |
| sll | $(rd) \leftarrow (rs1) \ll (rs2)$       |
| srl | $(rd) \leftarrow (rs1) \gg (rs2)$ ，逻辑右移 |
| sra | $(rd) \leftarrow (rs1) \gg (rs2)$ ，算术右移 |

#### I 型

|      |                                                                                              |
|------|----------------------------------------------------------------------------------------------|
| addi | $(rd) \leftarrow (rs1) + sext(imm)$                                                          |
| andi | $(rd) \leftarrow (rs1) \& sext(imm)$                                                         |
| ori  | $(rd) \leftarrow (rs1)   sext(imm)$                                                          |
| xori | $(rd) \leftarrow (rs1) \wedge sext(imm)$                                                     |
| slli | $(rd) \leftarrow (rs1) \ll shamt$                                                            |
| srli | $(rd) \leftarrow (rs1) \gg shamt$ ，逻辑右移                                                      |
| srai | $(rd) \leftarrow (rs1) \gg shamt$ ，算术右移                                                      |
| lw   | $(rd) \leftarrow sext(Mem[(rs1) + sext(offset)][31:0])$                                      |
| jalr | $t \leftarrow (pc) + 4; (pc) \leftarrow ((rs1) + sext(offset)) \& \sim 1; (rd) \leftarrow t$ |

#### S 型

|    |                                                    |
|----|----------------------------------------------------|
| sw | $Mem[(rs1) + sext(offset)] \leftarrow (rs2)[31:0]$ |
|----|----------------------------------------------------|

## 多周期

## R 型

|     |                                          |
|-----|------------------------------------------|
| add | $(rd) \leftarrow (rs1) + (rs2)$          |
| sub | $(rd) \leftarrow (rs1) - (rs2)$          |
| and | $(rd) \leftarrow (rs1) \& (rs2)$         |
| or  | $(rd) \leftarrow (rs1)   (rs2)$          |
| xor | $(rd) \leftarrow (rs1) \wedge (rs2)$     |
| sll | $(rd) \leftarrow (rs1) \ll (rs2)$        |
| srl | $(rd) \leftarrow (rs1) \gg (rs2)$ , 逻辑右移 |
| sra | $(rd) \leftarrow (rs1) \gg (rs2)$ , 算术右移 |

## I 型

|      |                                                                                                            |
|------|------------------------------------------------------------------------------------------------------------|
| addi | $(rd) \leftarrow (rs1) + \text{sext}(\text{imm})$                                                          |
| andi | $(rd) \leftarrow (rs1) \& \text{sext}(\text{imm})$                                                         |
| ori  | $(rd) \leftarrow (rs1)   \text{sext}(\text{imm})$                                                          |
| xori | $(rd) \leftarrow (rs1) \wedge \text{sext}(\text{imm})$                                                     |
| slli | $(rd) \leftarrow (rs1) \ll \text{shamt}$                                                                   |
| srli | $(rd) \leftarrow (rs1) \gg \text{shamt}$ , 逻辑右移                                                            |
| srai | $(rd) \leftarrow (rs1) \gg \text{shamt}$ , 算术右移                                                            |
| lw   | $(rd) \leftarrow \text{sext}(\text{Mem}[(rs1) + \text{sext}(\text{offset})][31:0])$                        |
| jalr | $t \leftarrow (pc) + 4; (pc) \leftarrow ((rs1) + \text{sext}(\text{offset})) \& \sim 1; (rd) \leftarrow t$ |

## S 型

|    |                                                                         |
|----|-------------------------------------------------------------------------|
| sw | $\text{Mem}[(rs1) + \text{sext}(\text{offset})] \leftarrow (rs2)[31:0]$ |
|----|-------------------------------------------------------------------------|

## B 型

|     |                                                                                   |
|-----|-----------------------------------------------------------------------------------|
| beq | if $((rs1) = (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$            |
| bne | if $((rs1) \neq (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$         |
| blt | if $((rs1) < (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$ , 有符号比较    |
| bge | if $((rs1) \geq (rs2)) (pc) \leftarrow (pc) + \text{sext}(\text{offset})$ , 有符号比较 |

## U 型

|     |                                                         |
|-----|---------------------------------------------------------|
| lui | $(rd) \leftarrow \text{sext}(\text{imm}[31:12] \ll 12)$ |
|-----|---------------------------------------------------------|

## J 型

|     |                                                                               |
|-----|-------------------------------------------------------------------------------|
| jal | $(rd) \leftarrow (pc) + 4; (pc) \leftarrow (pc) + \text{sext}(\text{offset})$ |
|-----|-------------------------------------------------------------------------------|

## 设计的主要特色（除基本要求以外的设计）

1. 分支预测功能：采用静态分支预测，默认不进行跳转。判断跳转在 EX 阶段，故而如果进行跳转，需要插入两条空指令。
2. 实现了结构清晰、可高度复用的总线外设：由于在设计时有意将 CPU、IROM、DRAM 分离，所以后续得到了很清晰的 IROM、CPU、BUS、DRAM、I/O 设备的结构，这样的结构使得代码能够极为方便的复用和理解，并可适用于单周期和流水线的两个上板测试。

## 资源使用、功耗数据截图（Post Implementation；含单周期、流水线 2 个截图）

老实说，这一个环节的必要性实在是难以恭维。

因为我们学的知识过于浅显，导致经过处理时序约束处理而产生的这些数据具有很低的参考价值。例如 Worst Negative Slack(wns) 表示一个时钟周期内最短的剩余时间，本来可以将其用于计算最大频率 ( $f_{MAX} = 1/(1/F - wns)$ )，但是由于没有处理时序导致其对于计算最大频率没有价值。经过测试，发现上板能够得到的最高频率比通过计算 wns 得到的理论最高频率高出太多（在上板达到最高频率时，wns 通常为较大的负数）。

各种测试都是以 trace 上板的工程来测试，这是因为计算器的汇编中的 load 和 save 指令太少，指令的类型也不全面，而 trace 的汇编就很合适。计算器上板能得到的最高频率也是相当不准的，例如在单周期 CPU 中，计算器上板可以达到 100MHz 的频率，而 trace 上板在 50MHz 时就已经失败。

时钟分频设置的有效位数仅限两位，设置的再精确，实际的输出频率也是不准的。例如设置 49.84375MHz，实际输出 50MHz，设置 49.375MHz，实际输出为 49.383MHz。故而没办法也没必要追求测试出精确的最大频率（即使是有两位的有效数字，也不需要频率设置的精确到两位），所以实验只包含有限的、粗略的几个点。

对于单周期 CPU，实验 25、30、35、40、45、50MHz 的频率；

对于流水线 CPU，实验 50、60、70、80、90、100MHz 的频率。

备注：在提升了频率之后，会出现按下 rst 才能正常跑完 trace 的情况；会出现按下 rst 之后出现不同结果的情况。

何为通过：按下 rst 后得到的结果总是唯一的 2500 0018。反例：2500 0008 和 2500 0018 各 50% 的概率，显然为不通过。

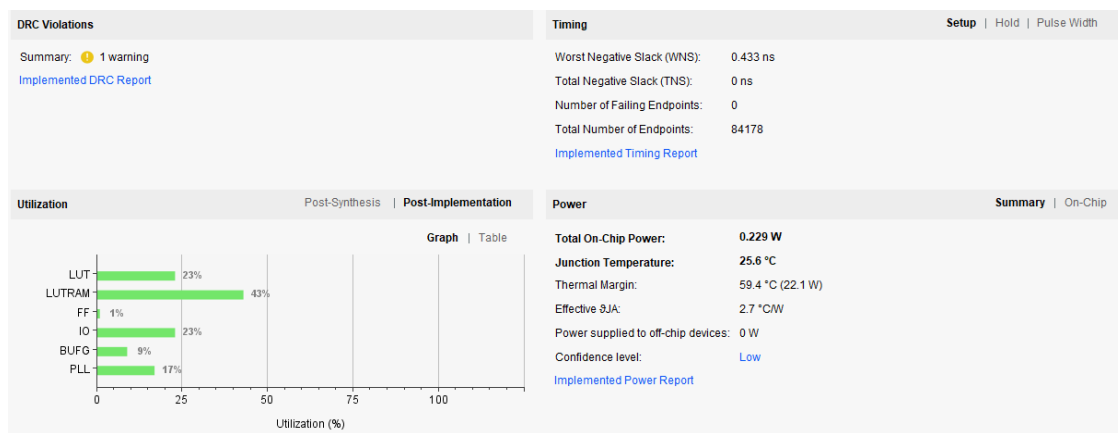
截图：选取 wns 为正数且尽量小的。

## 单周期 CPU

| 频率<br>(MHz) | wns       | 是否通过         | 备注                                                            |
|-------------|-----------|--------------|---------------------------------------------------------------|
| 25          | 4.718ns   | 是            |                                                               |
| 30          | 0.433ns   | 是            |                                                               |
| 35          | -0.181ns  | 是            |                                                               |
| 40          | -3.338ns  | 尚未按照<br>要求测试 |                                                               |
| 45          | -6.419ns  | 尚未按照<br>要求测试 |                                                               |
| 50          | -10.981ns | 否            | 第一次显示 0000 0000, 按下 rst 后大概率<br>显示 2500 0018, 小概率显示 2500 0008 |

很不幸的是，实验室的板子已经被收走了，无法继续上板实验，没能测出最大频率。

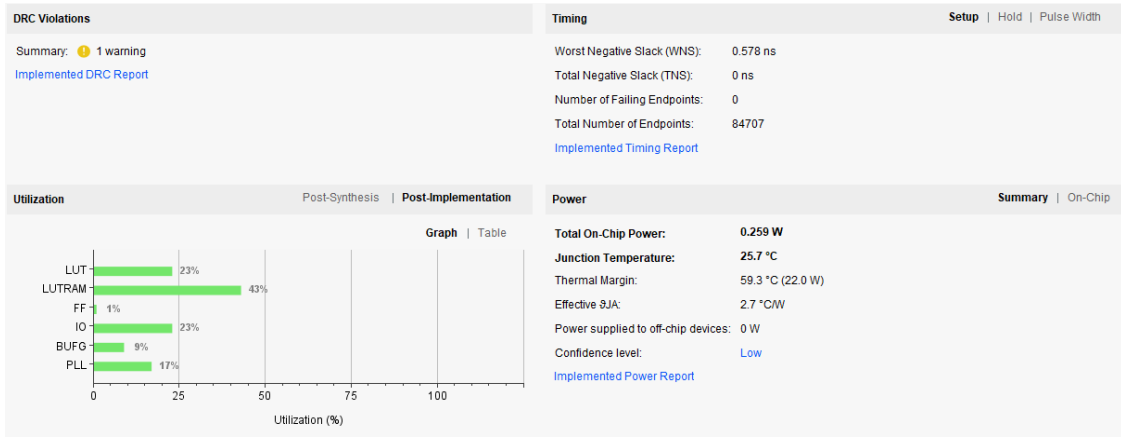
## 截图：(30MHz)



流水线 CPU

| 频率<br>(MHz) | wns      | 是否通过         | 备注                                                          |
|-------------|----------|--------------|-------------------------------------------------------------|
| 50          | 4.16ns   | 是            |                                                             |
| 60          | 0.578ns  | 尚未按照<br>要求测试 |                                                             |
| 70          | -0.132ns | 尚未按照<br>要求测试 |                                                             |
| 80          | -2.056ns | 尚未按照<br>要求测试 |                                                             |
| 90          | -2.781ns | 尚未按照<br>要求测试 |                                                             |
| 100         | -3.944ns | 否            | 第一次显示 0000 0000，按下 rst 后大概率<br>显示 2500 0018，小概率显示 2500 0008 |

截图：（60MHz）



Extra

trace 上板工程,单周期 10 - 50MHz 或者流水线 50 - 100MHz,在不加 clock.xdc 的条件下, 综合实现后首先 Timing 的数据都显示 NA, 其次会出现功耗失常。

所以不要忘记加 clock.xdc 了。

Power

Summary | On-Chip

Total On-Chip Power:

77.432 W (Junction temp exceeded!)

Junction Temperature:

125.0 °C

超过最大允许的结温。

Thermal Margin:

-147.2 °C (-54.4 W)

为降低结温,采用功耗优化策略来减少动态功耗,降低环境温度和/或使用散热器。参考UG807 来获取更多技术。

Effective  $\theta_{JA}$ :

2.7 °C/W

Power supplied to off-chip devices:

0 W

Confidence level:

Low

Implemented Power Report

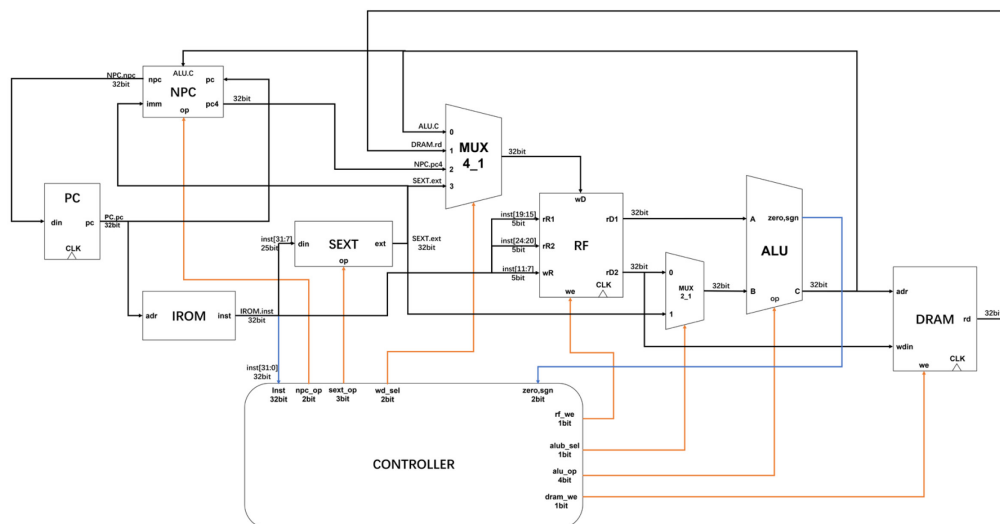
实验:

| 实验序号 | CPU     | 工程       | CPU 频率 | 是否有 clock.xdc | 功耗 |
|------|---------|----------|--------|---------------|----|
| 1    | 单周期 CPU | trace 上板 | 10MHz  | 无             | 失常 |
| 2    | 单周期 CPU | trace 上板 | 10MHz  | 有             | 正常 |
| 3    | 单周期 CPU | trace 上板 | 25MHz  | 无             | 失常 |
| 5    | 单周期 CPU | trace 上板 | 30MHz  | 无             | 失常 |
| 6    | 单周期 CPU | trace 上板 | 35MHz  | 无             | 失常 |
| 7    | 单周期 CPU | trace 上板 | 40MHz  | 无             | 失常 |
| 8    | 单周期 CPU | trace 上板 | 45MHz  | 无             | 失常 |
| 9    | 单周期 CPU | trace 上板 | 50MHz  | 无             | 失常 |
| 10   | 流水线 CPU | trace 上板 | 50MHz  | 无             | 失常 |
| 11   | 流水线 CPU | trace 上板 | 60MHz  | 无             | 失常 |
| 12   | 流水线 CPU | trace 上板 | 70MHz  | 无             | 失常 |
| 13   | 流水线 CPU | trace 上板 | 80MHz  | 无             | 失常 |
| 14   | 流水线 CPU | trace 上板 | 90MHz  | 无             | 失常 |
| 15   | 流水线 CPU | trace 上板 | 100MHz | 无             | 失常 |

# 1 单周期 CPU 设计与实现

## 1.1 单周期 CPU 整体框图

要求：无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，以及说明每个模块的功能含义。



模块说明：

### 1. 取指模块

- PC: 更新当前 pc 值
- NPC: 得到下一条指令的 pc 值
- IROM: 指令存储器, 根据 pc 值得到对应的指令

### 2. 译指/写回模块

- SEXT: 立即数扩展单元, 根据控制信号对指令中的立即数进行扩展
- RF: 寄存器堆, 可根据地址读出或写入对应寄存器数据
- MUX4\_1: 多路选择器, 选择写入 RF 的数据

### 3. 执行模块

- MUX2\_1: 选择器, 选择写入 ALU.B 端口的数据
- ALU: 运算器, 根据控制信号执行加、减、按位与、按位或、按位异或、逻辑右移、逻辑左移、算术右移运算

### 4. 存储模块

- DRAM: 数据存储器, 用于存储数据

### 5. 控制模块

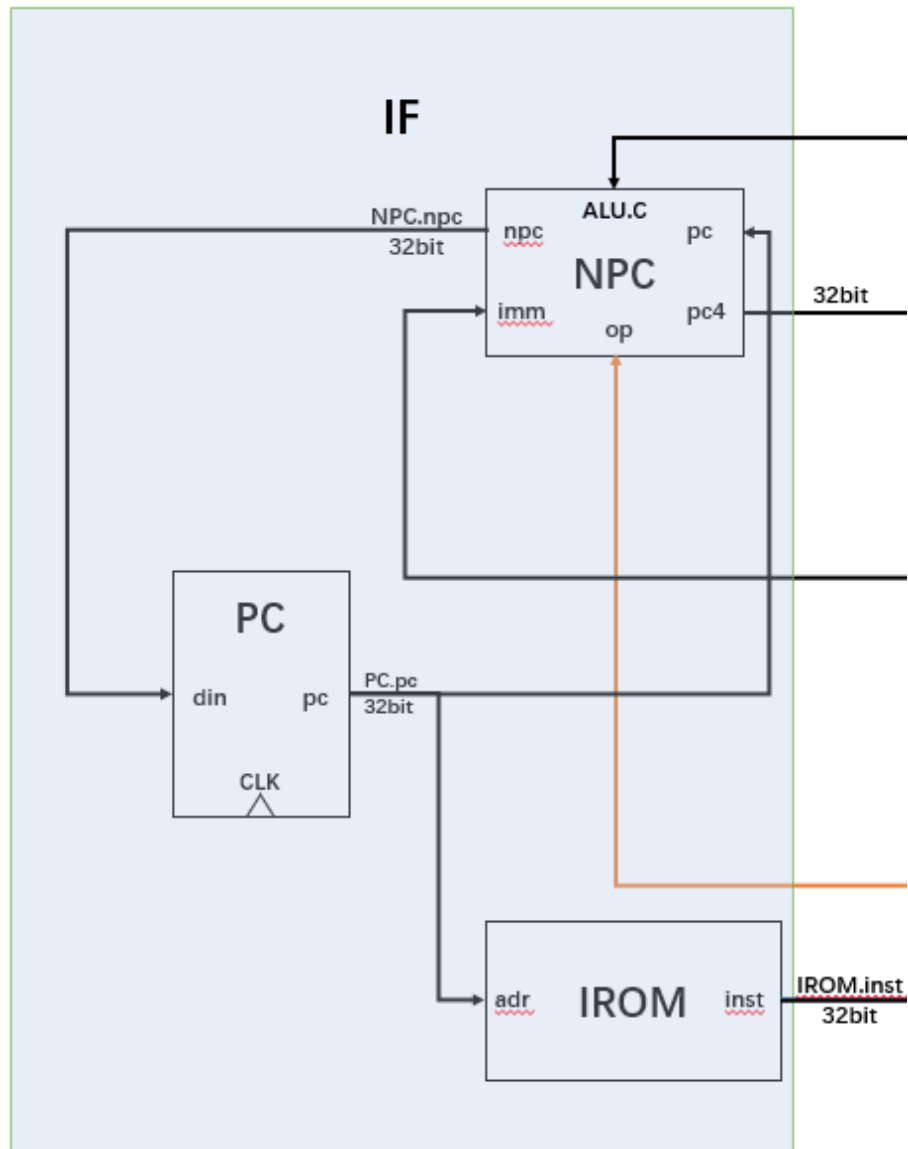
- CONTROLLER: 控制器, 根据指令生成各个器件的控制信号



## 1.2 单周期 CPU 模块详细设计

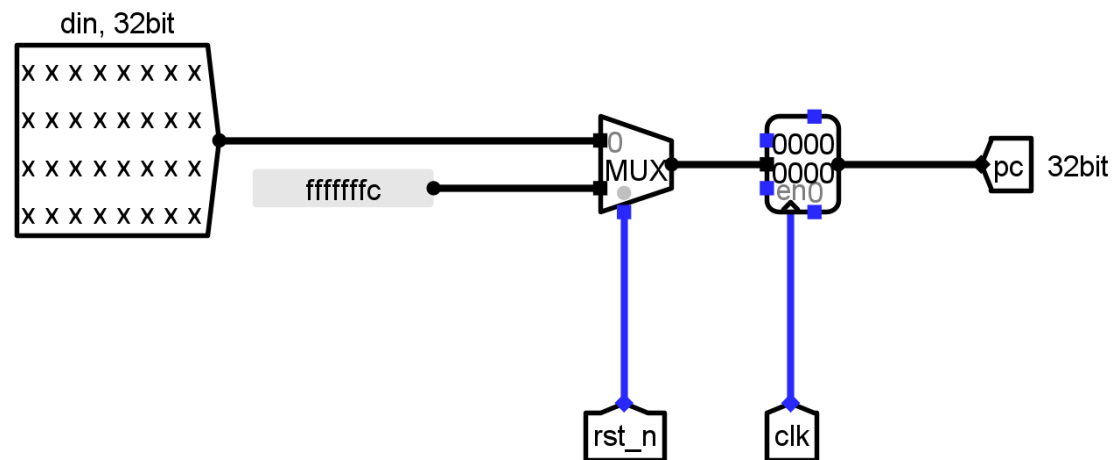
要求：画出各个模块的详细设计图，包含内部的子模块，以及关键性逻辑；标出子模块接口信号名、各信号线的信号名和位宽，并有详细的解释说明。

取指模块

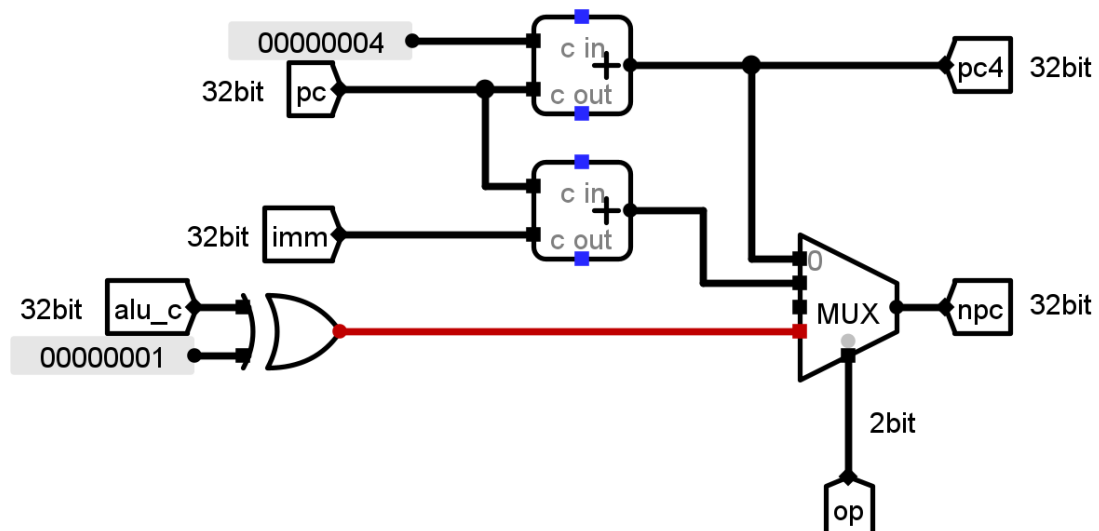


IROM: 指令存储器, 根据 pc 值得到对应的指令。

PC: 在时钟上升沿, 若  $\sim rst\_n$  为 0, 则寄存器更新为输入 pc 值 din; 若  $\sim rst\_n$  为 1, 则寄存器初始化为 -4。



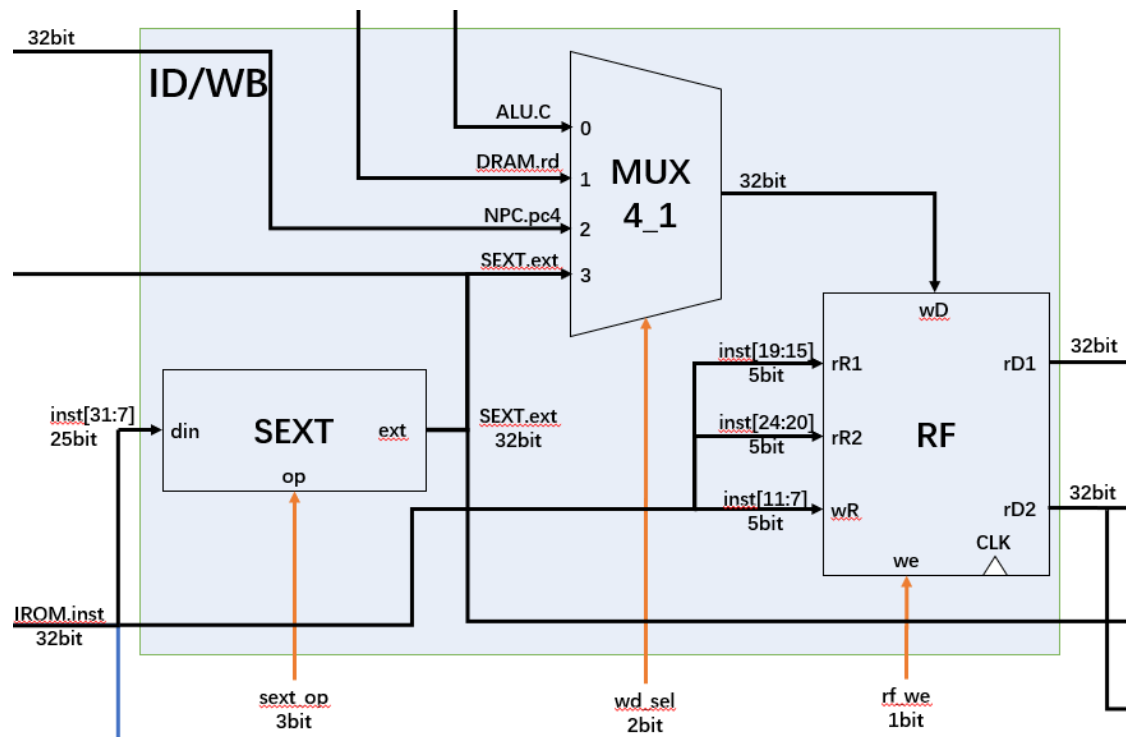
NPC: 根据 op 控制信号决定 npc 的输出。npc 共有三个来源, 分别为  $pc + 4$ 、 $pc + imm$ 、 $rd1 + imm$ 。同时, 输出后续计算所需数据:  $pc + 4$ 。



异或运算的用途: 使一个数的最低位为零。

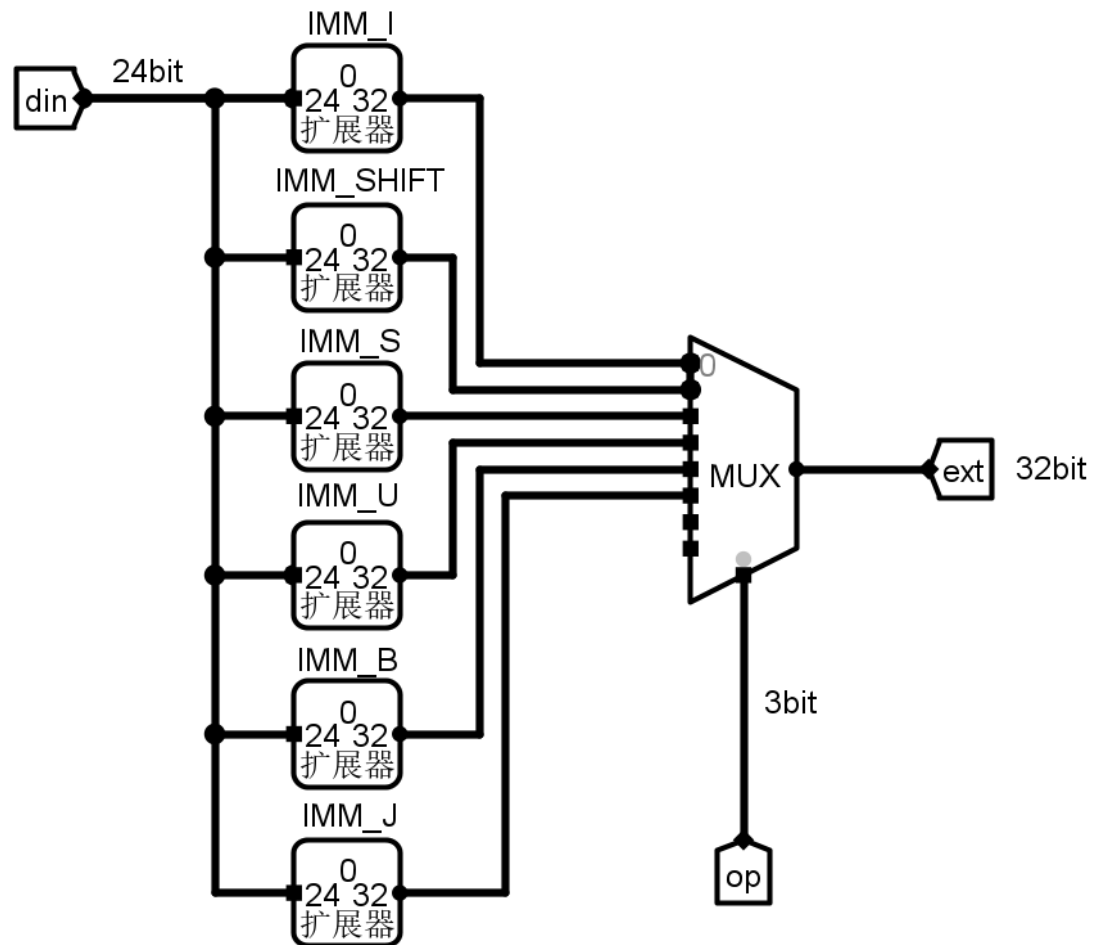
如: 使  $a$  的最低位为 0, 可以表示为:  $a \& \sim 1$ 。 $\sim 1$  的值为 1111 1111 1111 1110, 再按 "与" 运算, 最低位一定为 0。因为 " $\sim$ " 运算符的优先级比算术运算符、关系运算符、逻辑运算符和其他运算符都高。

## 译指/写回模块



SEXT: 立即数扩展单元, 根据控制信号对指令中的立即数进行扩展。

(数据的拼接过于复杂故图略)



RF:

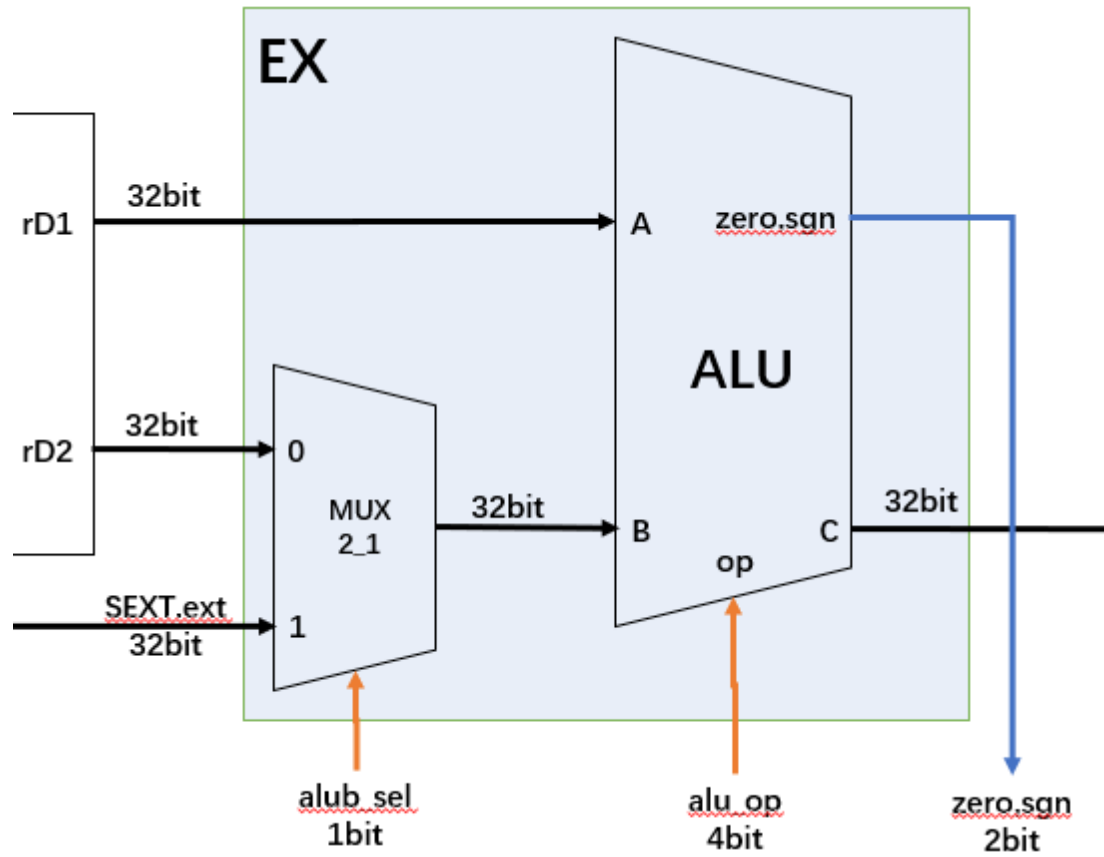
对 32 位指令 inst 进行解析, 得到两个读取地址 rR1、rR2 和一个写入地址 wR, 通过组合逻辑读出对应地址信号 rD1(RF[rR1])、rD2(RF[rR2])。

寄存器的读没有时钟限制, 可以直接读取。但是会在时钟上升沿写入, 并且对于写有一定的要求, 主要是为了达成零号寄存器读出的值为 0, 且不管写入零号寄存器的数据为多少, 总是只能写入 0 或者无法写入的目的:

- 1) 只有在写使能为 1 且写入寄存器号为不为 0 的情况下, 我们会把写入的数据 wD 写入对应的寄存器;
- 2) 此外情况, 只会完成想向零号寄存器里写入 0 的操作。

MUX4\_1: 多路选择器, 选择写入 RF 的数据。其写入数据通过四选一选择器对数据进行选择, 其写入数据包括: 运算器结果 ALU.C、存储器读出结果 DRAM.rd、pc + 4、拓展后的立即数 SEXText, 由 wb\_sel 信号决定写入数据, 由 rf\_we 信号决定是否写入。

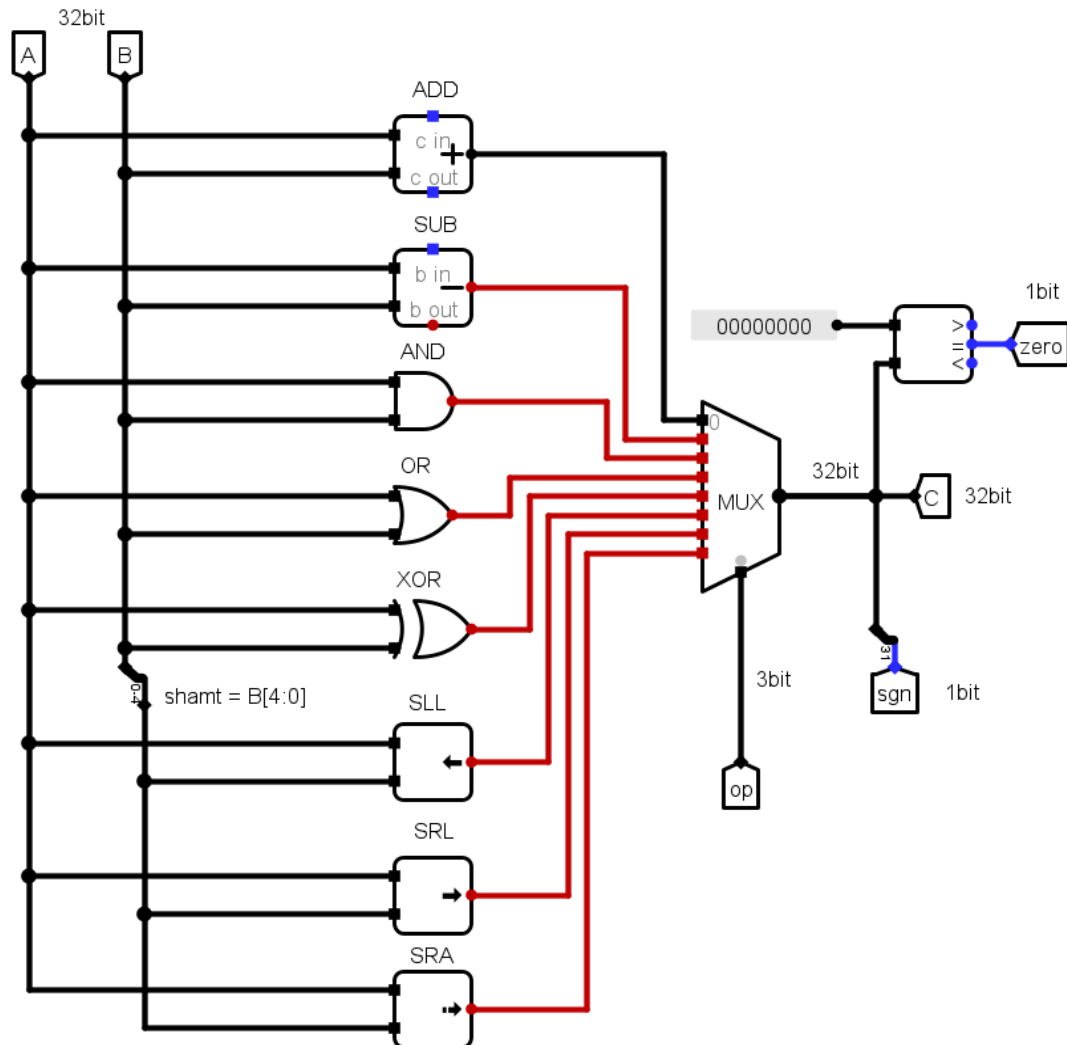
## 运算模块



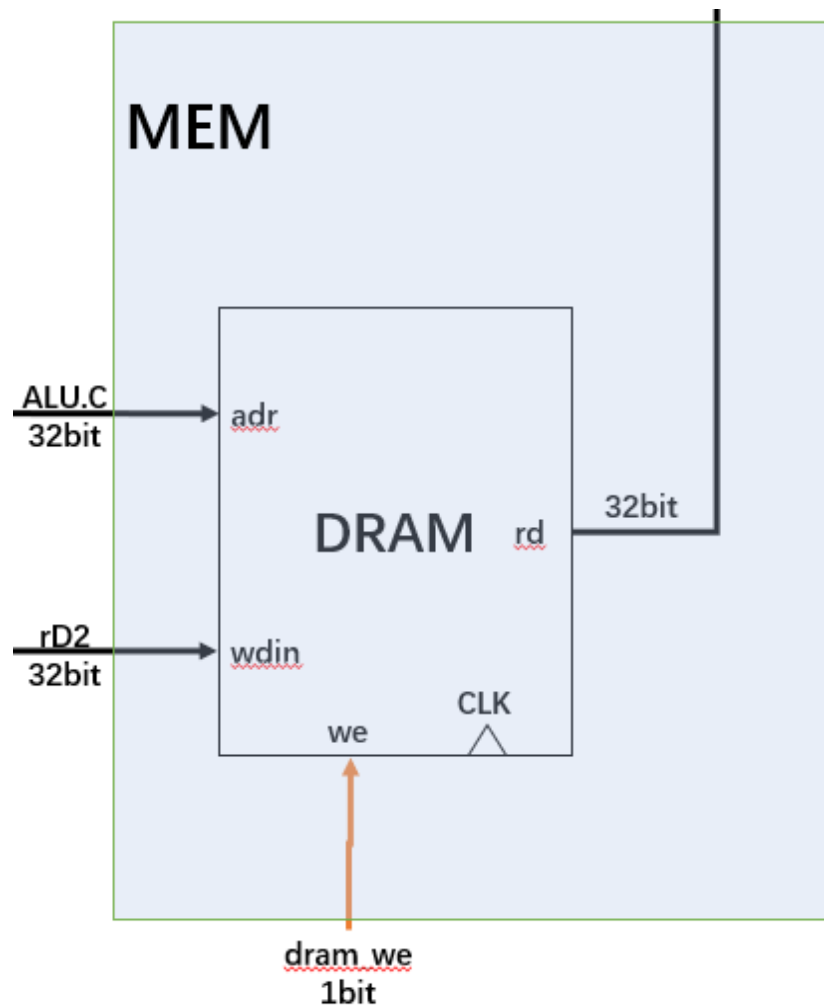
MUX2\_1: 选择器，选择写入 ALU.B 端口的数据。

ALU: 进行各种运算，并根据 `alu_op` 选择所需的运算结果。其中包括加、减、按位与、按位或、按位异或、逻辑右移、逻辑左移、算术右移运算模块。输出计算结果是否为 0，以及正负数的信号。

(SUB 计算的部分简化了，本来应该是  $A + (\sim B) + 1$ )

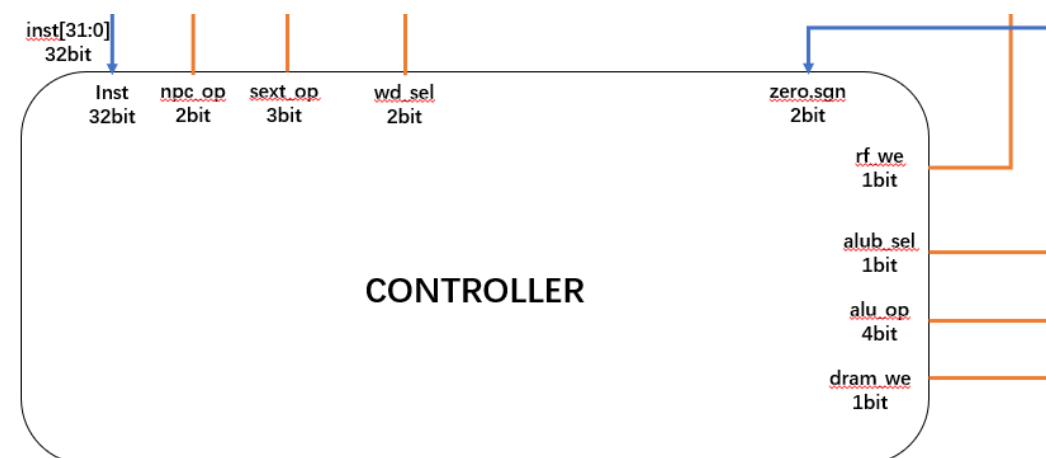


## 存储模块



DRAM: 数据存储器，用于存储数据。

## 控制器模块



CONTROLLER: 控制器，根据指令生成各个器件的控制信号。  
(具体图略)

### 1.3 单周期 CPU 仿真及结果分析

要求：包含逻辑运算、访存、分支跳转三类指令的仿真截图以及波形分析；每类指令的截图和分析中，至少包含 1 条具体指令；截图需包含信号名和关键信号。

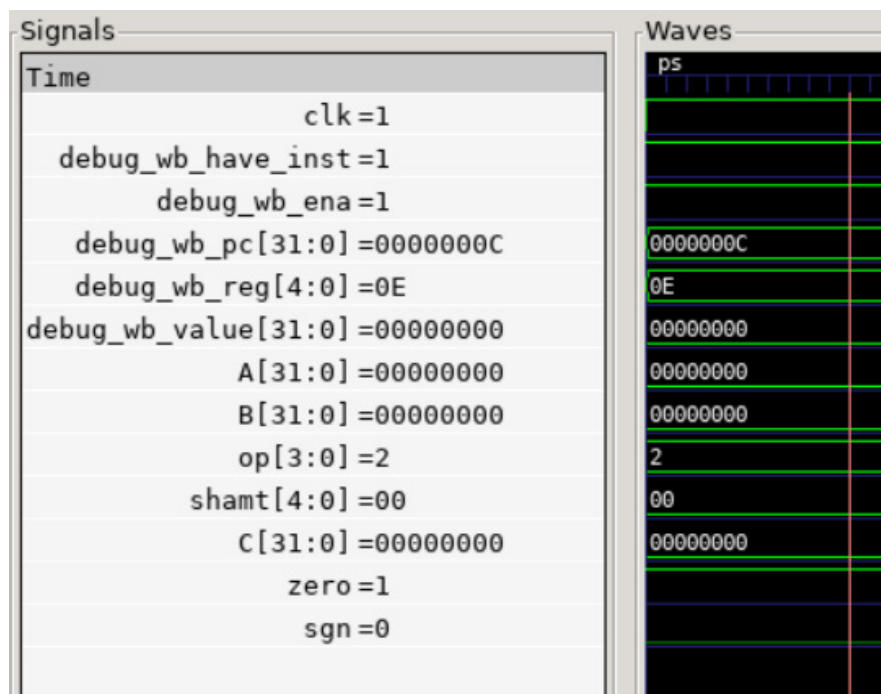
- add, from add.dump

00000004 <reset\_vector>:

```

4:  00000093      addi    x1,x0,0
8:  00000113      addi    x2,x0,0
c:  00208733      add x14,x1,x2
10: 00000393      addi    x7,x0,0
14: 00200193      addi    x3,x0,2
18: 4c771663      bne x14,x7,4e4 <fail>

```



pc = c, x1 = 0, x2 = 0, x14 = x1 + x2 = 0。



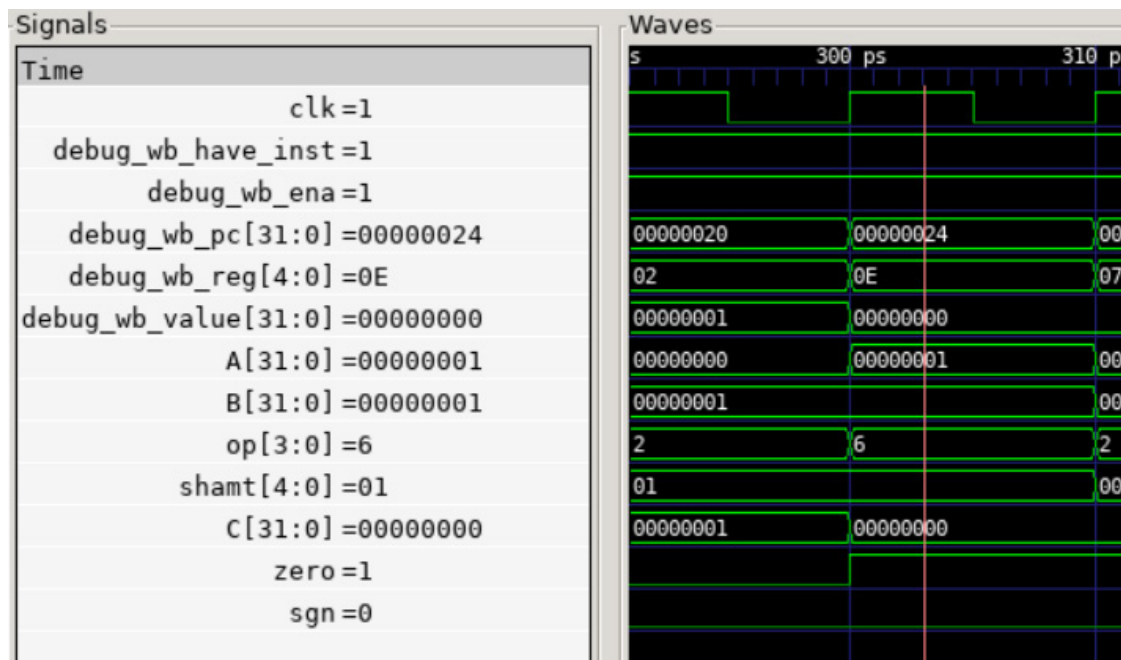
- sub, from sub.dump

0000001c <test\_3>:

```

1c:  00100093      addi    x1,x0,1
20:  00100113      addi    x2,x0,1
24:  40208733      sub x14,x1,x2
28:  00000393      addi    x7,x0,0
2c:  00300193      addi    x3,x0,3
30:  48771a63      bne x14,x7,4c4 <fail>

```



pc = 24, x1 = 1, x2 = 1, x14 = x1 - x2 = 0。

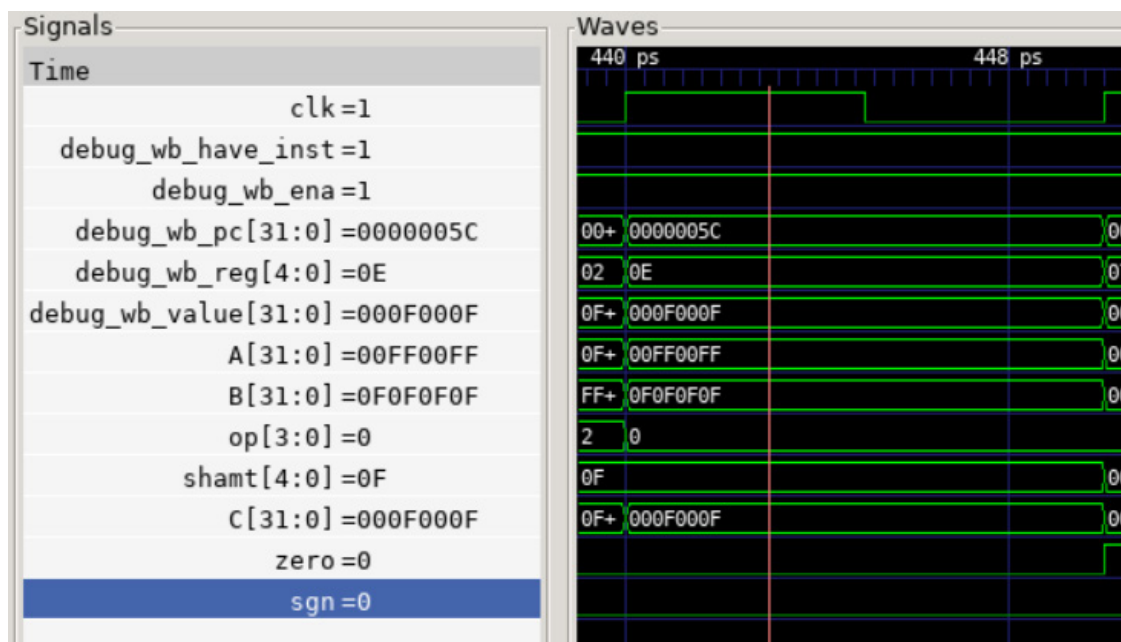
- and, from and.dump

0000004c &lt;test\_4&gt;:

```

4c:  00ff00b7      lui x1,0xff0
50:  0ff08093      addi x1,x1,255 # ff00ff <_end+0xfe0ff>
54:  0f0f1137      lui x2,0xf0f1
58:  f0f10113      addi x2,x2,-241 # f0f0f0f <_end+0xf0eef0f>
5c:  0020f733      and x14,x1,x2
60:  000f03b7      lui x7,0xf0
64:  00f38393      addi x7,x7,15 # f000f <_end+0xee00f>
68:  00400193      addi x3,x0,4
6c:  44771863      bne x14,x7,4bc <fail>

```



pc = 5c, x1 = 00ff 00ff, x2 = 0f0f 0f0f, x14 = x1 & x2 = 000f 000f.

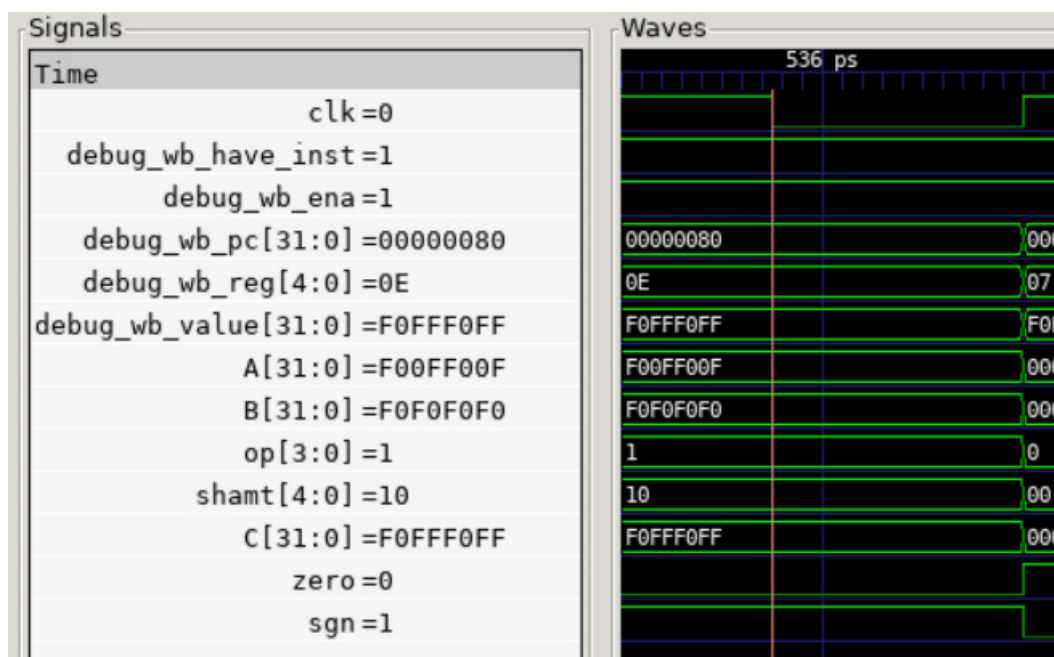
- or, from or.dump

00000070 &lt;test\_5&gt;:

```

70:  f00ff0b7      lui x1,0xf00ff
74:  00f08093      addi x1,x1,15 # f00ff00f <_end+0xf00fd00f>
78:  f0f0f137      lui x2,0xf0f0f
7c:  0f010113      addi x2,x2,240 # f0f0f0f0 <_end+0xf0f0d0f0>
80:  0020e733      or x14,x1,x2
84:  f0fff3b7      lui x7,0xf0fff
88:  0ff38393      addi x7,x7,255 # f0fff0ff <_end+0xf0ffd0ff>
8c:  00500193      addi x3,x0,5
90:  42771c63      bne x14,x7,4c8 <fail>

```



pc = 80, x1 = f00f f00f, x2 = f0f0 f0f0, x14 = x1 | x2 = f0ff f0ff.

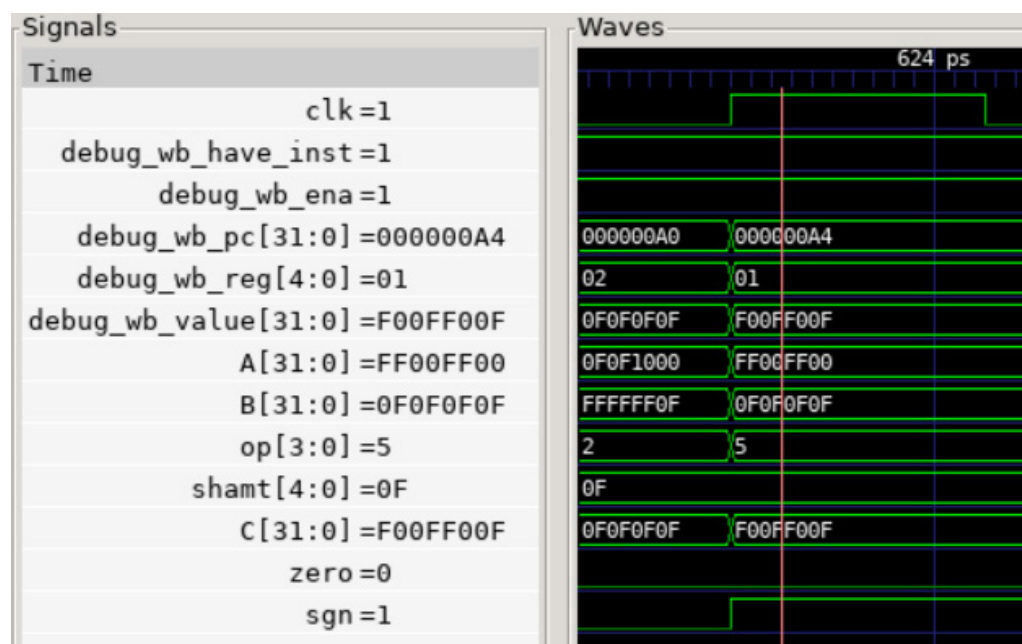
- xor, from xor.dump

00000094 &lt;test\_6&gt;:

```

94:  ff0100b7      lui x1,0xff010
98:  f0008093      addi x1,x1,-256 # ff00ff00 <_end+0xff00df00>
9c:  0f0f1137      lui x2,0xf0f1
a0:  f0f10113      addi x2,x2,-241 # f0f0f0f <_end+0xf0eef0f>
a4:  0020c0b3      xor x1,x1,x2
a8:  f00ff3b7      lui x7,0xf00ff
ac:  00f38393      addi x7,x7,15 # f00ff00f <_end+0xf00fd00f>
b0:  00600193      addi x3,x0,6
b4:  40709863      bne x1,x7,4c4 <fail>

```



pc = a4, x1 = ff00 ff00, x2 = 0f0f 0f0f, x1 = x1  $\oplus$  x2 = f00f f00f.

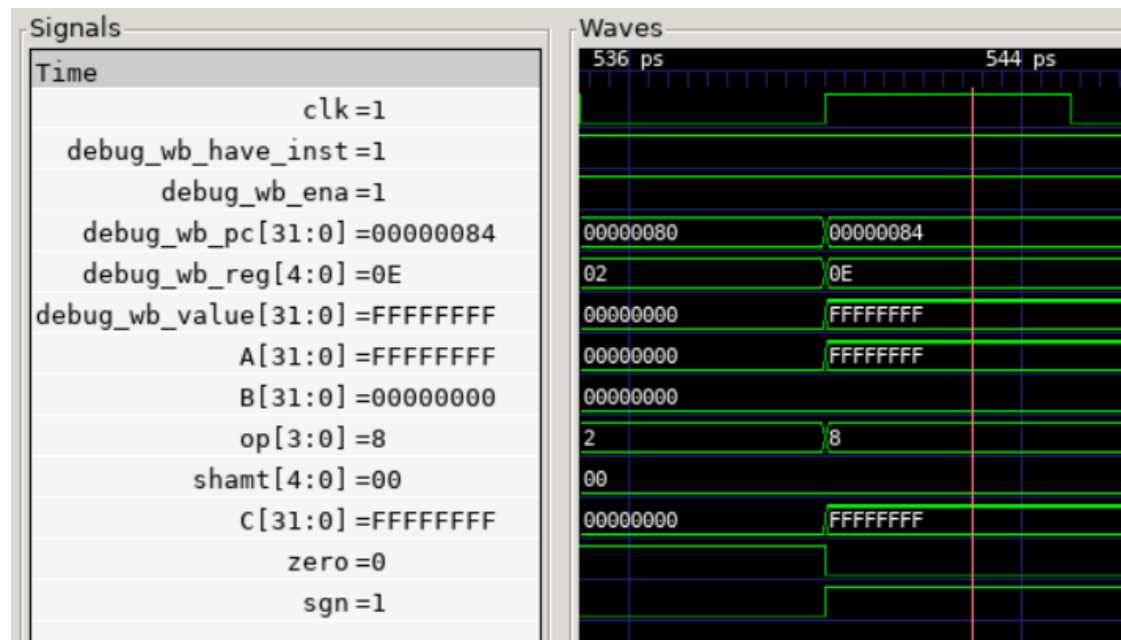
- sll, from sll.dump

0000007c &lt;test\_7&gt;:

```

7c:  fff00093      addi    x1,x0,-1
80:  00000113      addi    x2,x0,0
84:  00209733      sll x14,x1,x2
88:  fff00393      addi    x7,x0,-1
8c:  00700193      addi    x3,x0,7
90:  4c771263      bne x14,x7,554 <fail>

```



pc = 84, x1 = ffff ffff, x2 = 0, x14 = x1 << x2 = ffff ffff << 0 = ffff ffff.

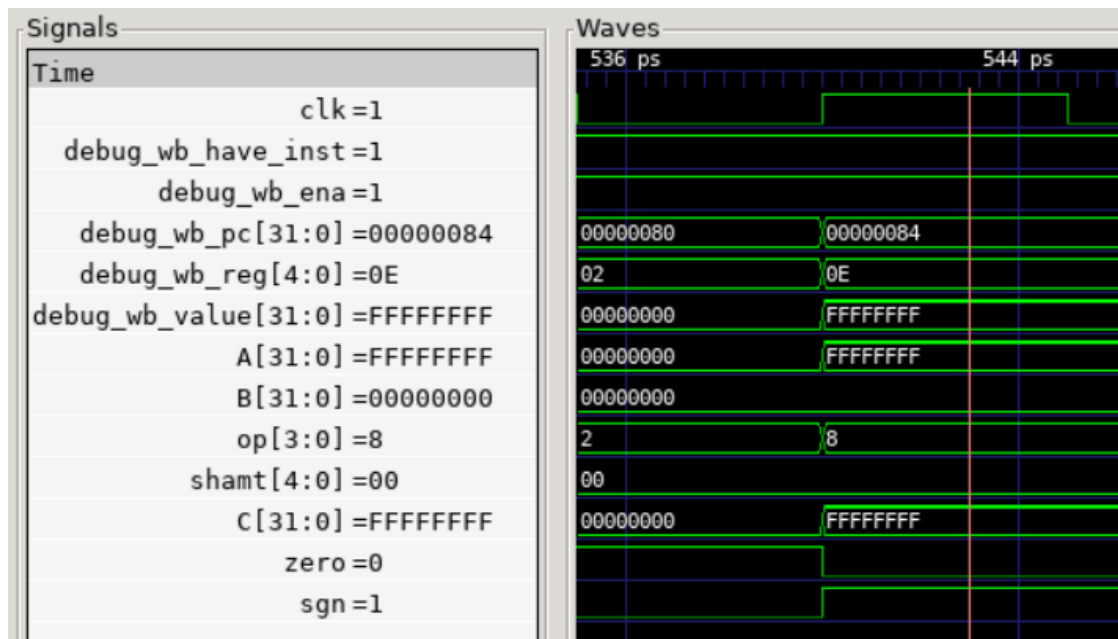
- sra, from sra.dump

000000a0 <test\_8>:

```

a0: 800000b7      lui x1,0x80000
a4: fff08093      addi x1,x1,-1 # 7ffffff <_end+0x7fffdfff>
a8: 00100113      addi x2,x0,1
ac: 4020d733      sra x14,x1,x2
b0: 400003b7      lui x7,0x40000
b4: fff38393      addi x7,x7,-1 # 3ffffff <_end+0x3fffdfff>
b8: 00800193      addi x3,x0,8
bc: 4e771263      bne x14,x7,5a0 <fail>

```



pc = ac, x1 = 7fff ffff, x2 = 1, x14 = x1 >>> x2 = 7fff ffff >>> 1 = 3fff ffff。

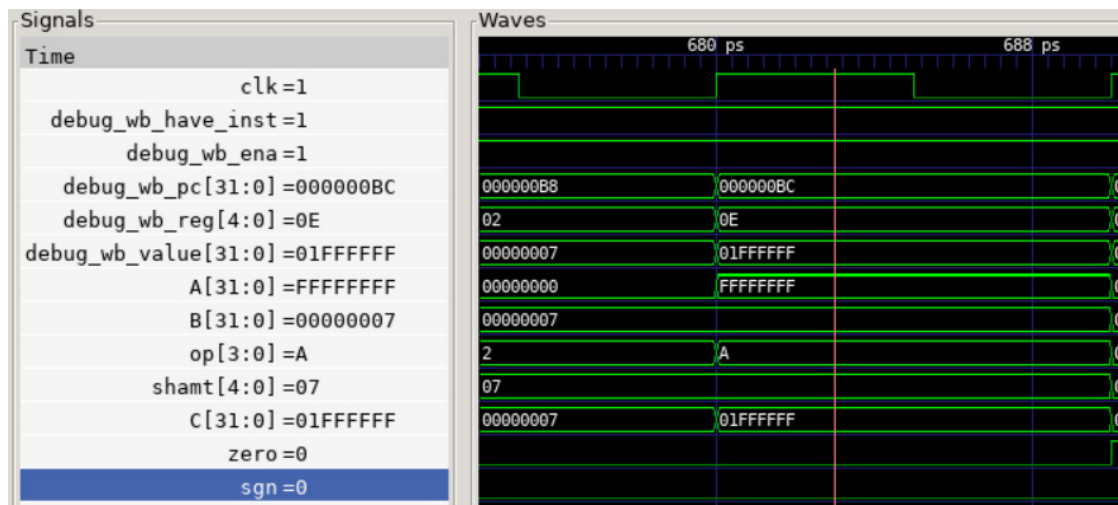
- srl, from srl.dump

000000b4 &lt;test\_9&gt;:

```

b4:  fff00093      addi    x1,x0,-1
b8:  00700113      addi    x2,x0,7
bc:  0020d733      srl x14,x1,x2
c0:  020003b7      lui x7,0x2000
c4:  fff38393      addi    x7,x7,-1 # 1ffffff <_end+0x1ffdfff>
c8:  00900193      addi    x3,x0,9
cc:  4a771e63      bne x14,x7,588 <fail>

```



pc = bc, x1 = ffff ffff, x2 = 7, x14 = x1 >> x2 = ffff ffff >> 7 = 01ff ffff.

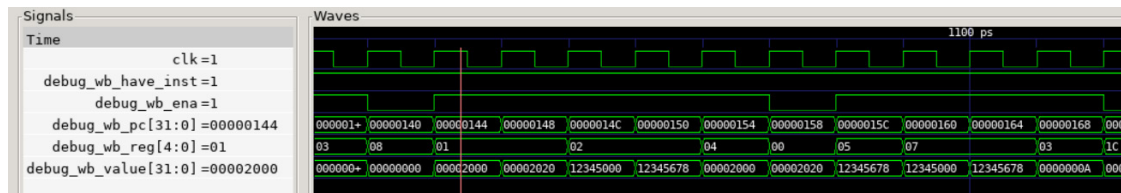
- 访存指令，from sw.dump

00000144 <test\_10>:

```

144: 000020b7      lui x1,0x2
148: 02008093      addi x1,x1,32 # 2020 <tdat9>
14c: 12345137      lui x2,0x12345
150: 67810113      addi x2,x2,1656 # 12345678 <_end+0x12343648>
154: fe008213      addi x4,x1,-32
158: 02222023      sw x2,32(x4) # 20 <reset_vector+0x1c>
15c: 0000a283      lw x5,0(x1)
160: 123453b7      lui x7,0x12345
164: 67838393      addi x7,x7,1656 # 12345678 <_end+0x12343648>
168: 00a00193      addi x3,x0,10
16c: 30729e63      bne x5,x7,488 <fail>

```



$x1 = 0000\ 2020$ ,  $x2 = 1234\ 5678$ ,  $x4 = 0000\ 2000$ ;

把  $x2$  的值存到基地址  $0000\ 2000$ ，偏移  $32$  的地址，即  $0000\ 2020$ ;

读取基地址  $0000\ 2020$ ，偏移  $0$  的地址，即  $0000\ 2020$  到  $x5$  中。

那么  $x5 = 12345678$ 。

$x7 = 12345678$ ,  $x5 == x7$ ，故而不满足 `bne`，不跳转，可以注意到  $pc = 164$  后是  $pc = 168$ ，故没有发生跳转，对应正确。



- jal, from jal.dump

00000000 <\_start>:

0: 0040006f jal x0,4 <reset\_vector>

00000004 <reset\_vector>:

4: 00200193 addi x3,x0,2

8: 00000093 addi x1,x0,0

c: 0100026f jal x4,1c <target\_2>

00000010 <linkaddr\_2>:

10: 00000013 addi x0,x0,0

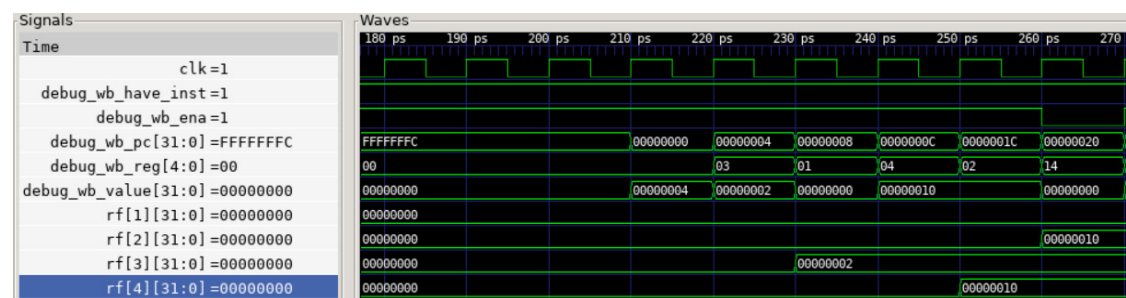
14: 00000013 addi x0,x0,0

18: 03c0006f jal x0,54 <fail>

0000001c <target\_2>:

1c: 01000113 addi x2,x0,16

20: 02411a63 bne x2,x4,54 <fail>



第一个 jal 指令没有什么作用，从可以正常到 pc = 4 看出没有问题；

第二个 jal 指令会向 x4 存储 pc + 4，即 10，跳转到 pc = 1c，也没有问题。

- jalr, from jalr.dump

00000004 <reset\_vector>:

```

4:  00200193      addi    x3,x0,2
8:  00000293      addi    x5,x0,0
c:  01800313      addi    x6,x0,24
10: 000302e7      jalr    x5,0(x6)

```

00000014 <linkaddr\_2>:

```

14: 0c40006f      jal x0,d8 <fail>

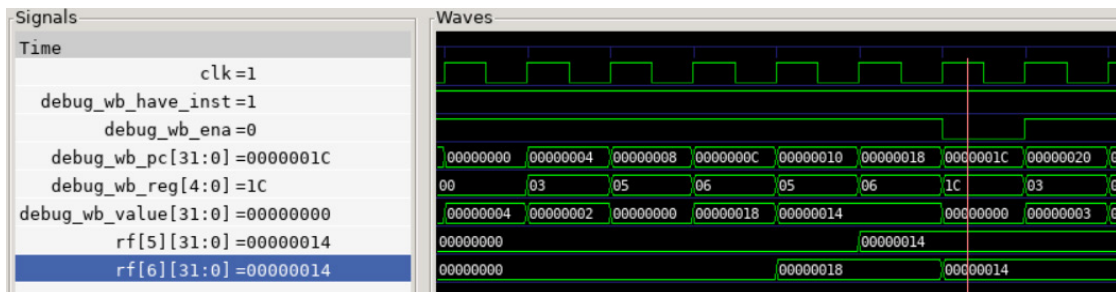
```

00000018 <target\_2>:

```

18: 01400313      addi    x6,x0,20
1c: 0a629e63      bne x5,x6,d8 <fail>

```



x6 = 18(16 进制)

jalr 指令使 x5 = 14(16 进制), 跳转到 pc = 18(16 进制), 没问题;

x6 = 14(16 进制)

x5 == x6, 不满足 bne, 不跳转, 没问题。

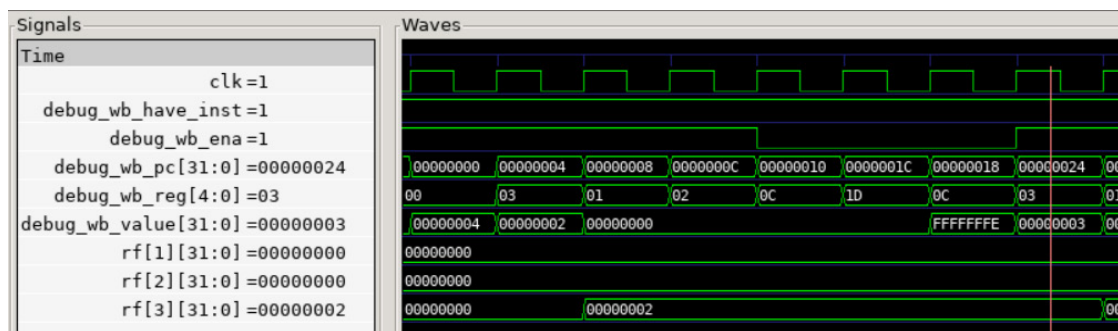
- beq、bne, from beq.dump

00000004 <reset\_vector>:

```

4:  00200193      addi    x3,x0,2
8:  00000093      addi    x1,x0,0
c:  00000113      addi    x2,x0,0
10: 00208663      beq x1,x2,1c <reset_vector+0x18>
14: 2a301863      bne x0,x3,2c4 <fail>
18: 00301663      bne x0,x3,24 <test_3>
1c: fe208ee3      beq x1,x2,18 <reset_vector+0x14>
20: 2a301263      bne x0,x3,2c4 <fail>

```



x3 = 2, x1 = 0, x2 = 0;

pc = 10: x1 == x2, 满足 beq, 跳转到 pc = 1c;

pc = 1c: x1 == x2, 满足 beq, 跳转到 pc = 18;

pc = 18: x0 != x3, 满足 bne, 跳转到 pc = 24。

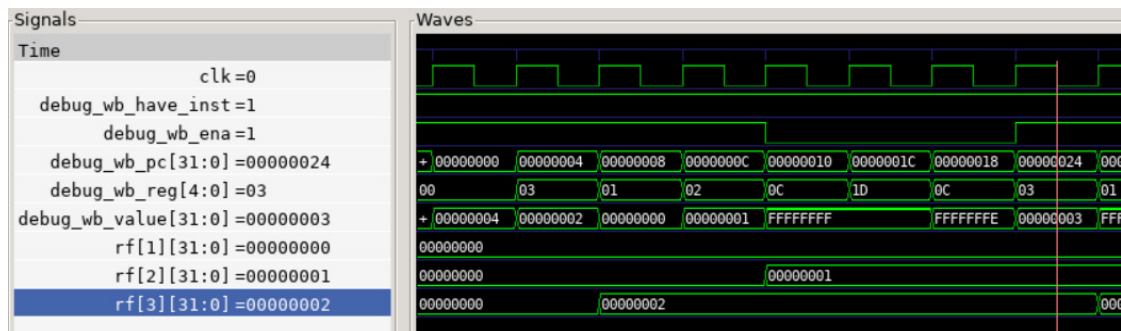
- blt

00000004 <reset\_vector>:

```

4:  00200193      addi    x3,x0,2
8:  00000093      addi    x1,x0,0
c:  00100113      addi    x2,x0,1
10: 0020c663      blt x1,x2,1c <reset_vector+0x18>
14: 2a301863      bne x0,x3,2c4 <fail>
18: 00301663      bne x0,x3,24 <test_3>
1c: fe20cee3      blt x1,x2,18 <reset_vector+0x14>
20: 2a301263      bne x0,x3,2c4 <fail>

```



$x3 = 2$ ,  $x1 = 0$ ,  $x2 = 1$ ;

pc = 10:  $x1 < x2$ , 满足 blt, 跳转到 pc = 1c;

pc = 1c:  $x1 < x2$ , 满足 blt, 跳转到 pc = 18;

pc = 18:  $x0 \neq x3$ , 满足 bne, 跳转到 pc = 24。

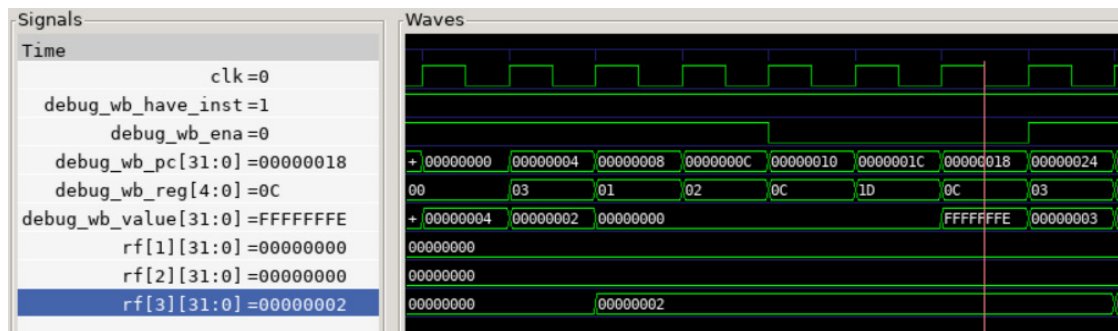
- bge

00000004 <reset\_vector>:

```

4:  00200193      addi    x3,x0,2
8:  00000093      addi    x1,x0,0
c:  00000113      addi    x2,x0,0
10: 0020d663      bge x1,x2,1c <reset_vector+0x18>
14: 30301863      bne x0,x3,324 <fail>
18: 00301663      bne x0,x3,24 <test_3>
1c: fe20dee3      bge x1,x2,18 <reset_vector+0x14>
20: 30301263      bne x0,x3,324 <fail>

```



$x3 = 2$ ,  $x1 = 0$ ,  $x2 = 0$ ;

pc = 10:  $x1 \geq x2$ , 满足 bge, 跳转到 pc = 1c;

pc = 1c:  $x1 \geq x2$ , 满足 bge, 跳转到 pc = 18;

pc = 18:  $x0 \neq x3$ , 满足 bne, 跳转到 pc = 24。

## 2 流水线 CPU 设计与实现

### 2.1 流水线的划分

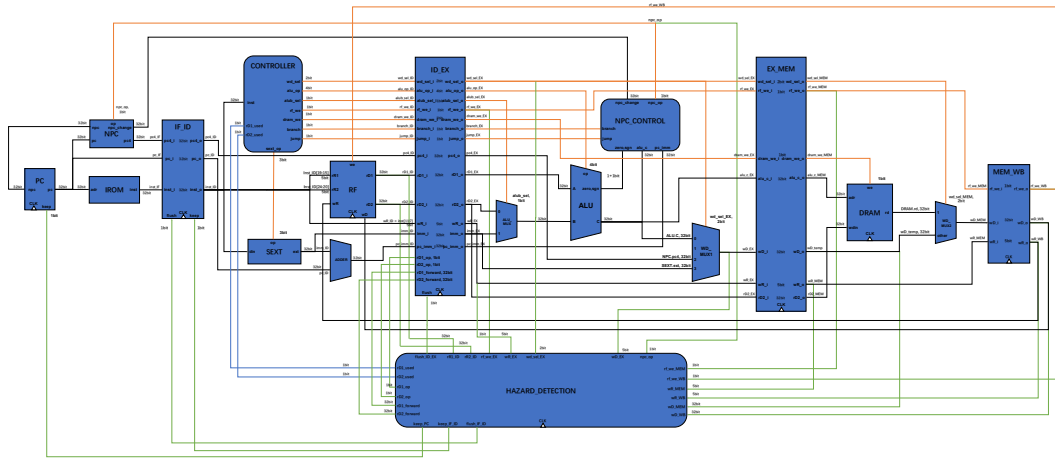
要求：画出流水线如何划分，说明每个流水级具备什么功能、需要完成哪些操作。

1. 取指阶段 (instruction fetch, 简称 IF 阶段)
  - (1) 更新 pc 值
  - (2) 得到当前 pc 对应的指令
  - (3) 可能会由于冒险检测器的控制信号而保持停顿
2. 译指阶段 (instruction decode, 简称 ID 阶段)
  - (1) 由控制器得到控制信号
  - (2) 得到寄存器堆中两个目标寄存器堆的值
  - (3) 得到对应类型的拓展的立即数
  - (4) 结合译指结果，在冒险检测器中判断当前是否需要数据前递以及停顿，并处理
3. 执行阶段 (execute, 简称 EX 阶段)
  - (1) ALU 根据控制信号以及操作数执行对应的运算
  - (2) 根据控制信号和 ALU 计算结果判断是否跳转以及跳转的 pc 值，传达给 IF 阶段的 NPC
  - (3) 完成写回寄存器堆数据的第一次选择 (处理)
4. 访存阶段 (memory access, 简称 MEM 阶段)
  - (1) 根据控制信号以及操作地址对存储器执行相应的访存操作
  - (2) 完成写回寄存器堆数据的第二次选择 (处理)
5. 写回阶段 (write back, 简称 WB 阶段)

根据控制信号向寄存器堆目标寄存器写入相应数据

## 2.2 流水线 CPU 整体框图

要求：无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，以及说明每个模块的功能含义。



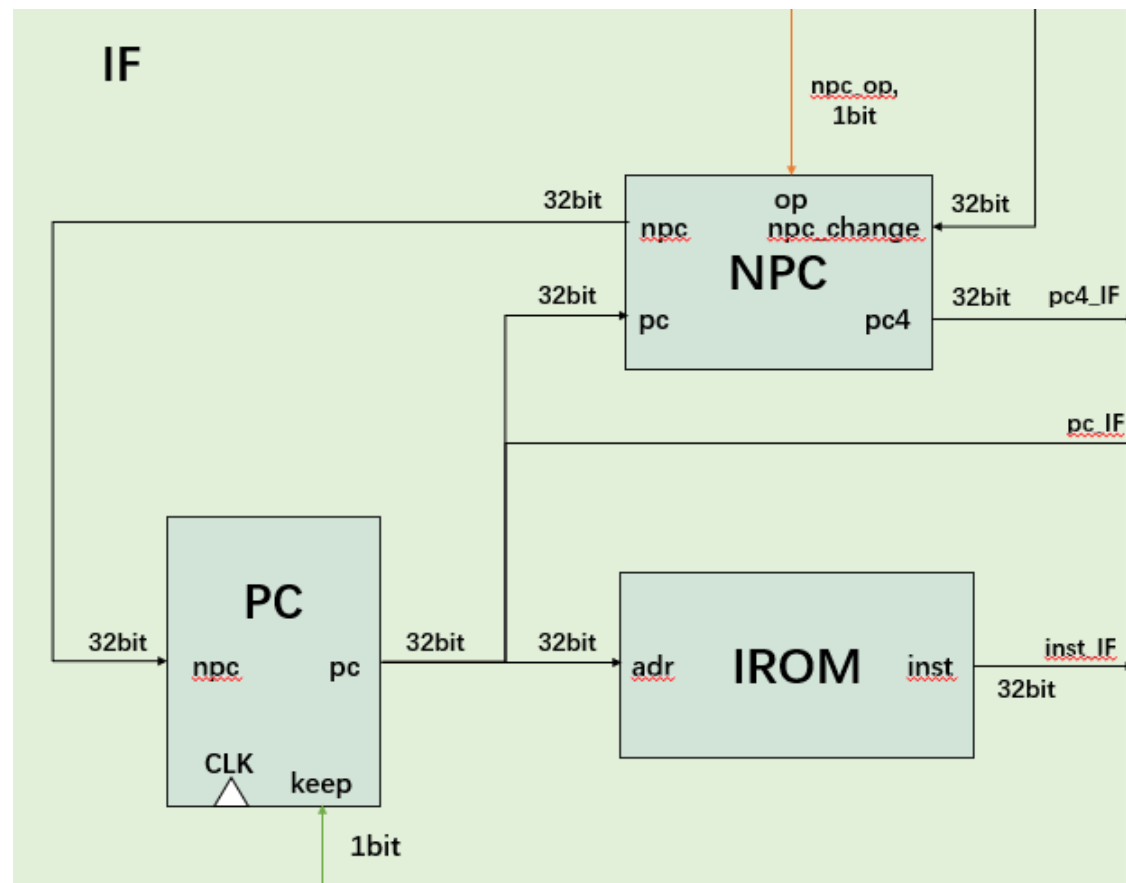
模块说明：

1. 取指模块
  - PC: 更新当前 pc 值
  - NPC: 得到下一条指令的 pc 值
  - IROM: 指令存储器, 根据 pc 值得到对应的指令
2. 译指模块
  - CONTROLLER: 控制器, 根据指令生成各个器件的控制信号
  - SEXT: 立即数扩展单元, 根据控制信号对指令中的立即数进行扩展
  - RF: 寄存器堆, 可根据地址读出或写入对应寄存器数据
3. 执行模块
  - ALU\_MUX: 选择器, 选择写入 ALU.B 端口的数据
  - ALU: 运算器, 根据控制信号执行加、减、按位与、按位或、按位异或、逻辑右移、逻辑左移、算术右移运算
  - NPC\_CONTROL: 根据控制信号和计算结果判断是否跳转以及跳转的 pc 值, 传达给 NPC
  - WD\_MUX1: 写回寄存器堆的数据 wD 的第一次选择
4. 存储模块
  - DRAM: 数据存储器, 用于存储数据
  - WD\_MUX2: 写回寄存器堆的数据 wD 的第二次选择
5. 流水线寄存器模块
  - REG\_IF\_ID、REG\_ID\_EX、REG\_EX\_MEM、REG\_MEM\_WB: 用于存储各阶段正在执行的指令信息
6. 冒险检测器模块
  - HAZARD\_DETECTION: 检测并处理数据冒险与控制冒险: 接收用于判断冒险的控制信号和数据, 发出前递、停顿、清除的控制信号和相关数据

## 2.3 流水线 CPU 模块详细设计

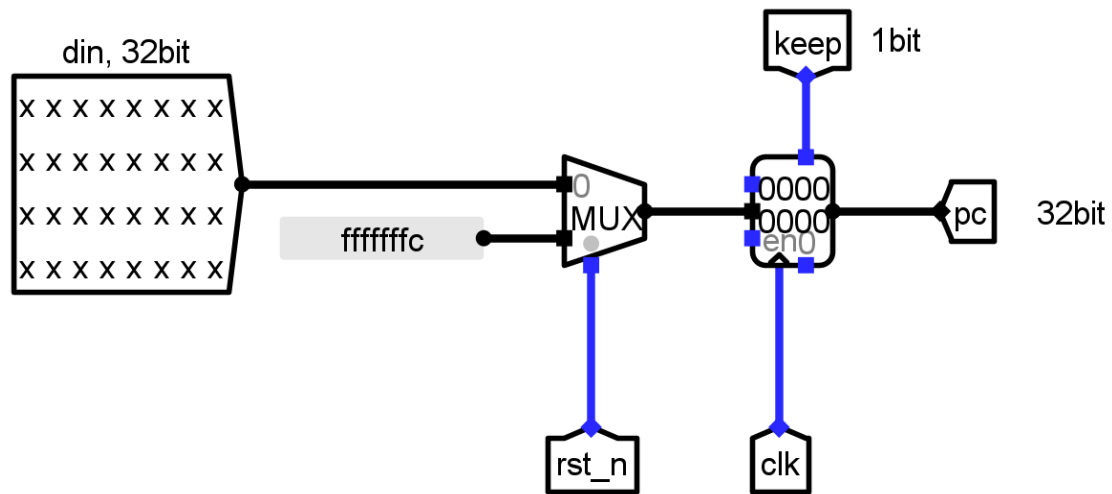
要求：画出各个模块的详细设计图，包含内部的子模块，以及关键性逻辑；标出子模块接口信号名、各信号线的信号名和位宽，并有详细的解释说明；此外，必须结合模块图，详细说明数据冒险、控制冒险的解决方法。

取指模块

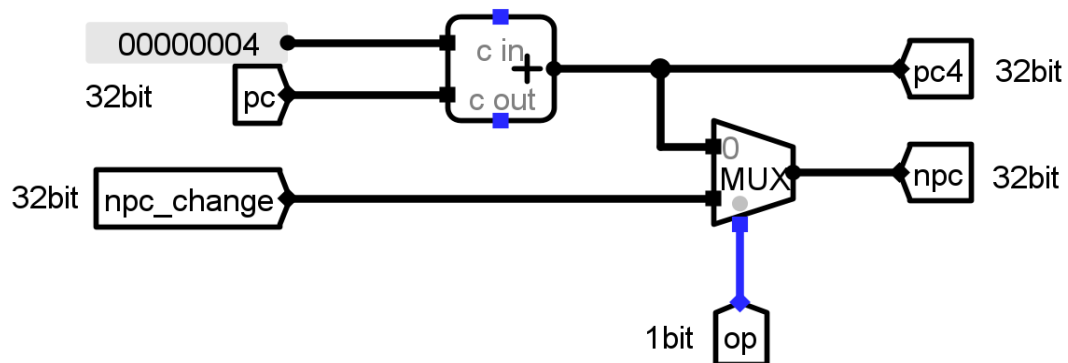




PC: 更新当前 pc 值; 当控制信号 keep 为 1 时保持寄存器内容在下一个时钟周期不变。

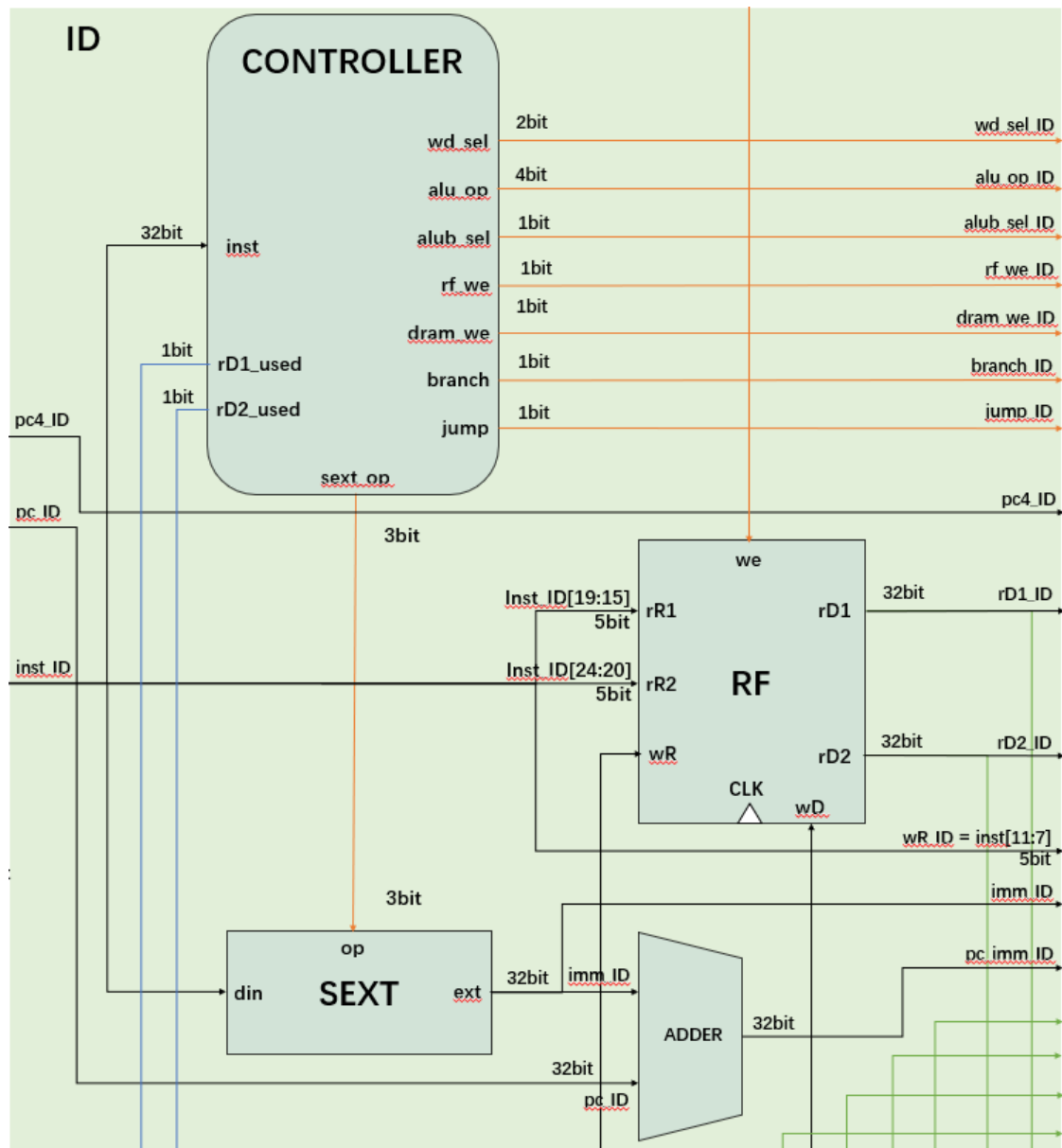


NPC: 得到下一条指令的 pc 值; 当控制信号 op 为 0 时, npc 输出为当前 pc + 4, op 为 1 时, npc 输出为 EX 阶段来的指定要跳转的 pc 值。故而 op 也是跳转的标志。



IROM 模块与单周期中相应模块原理相同。

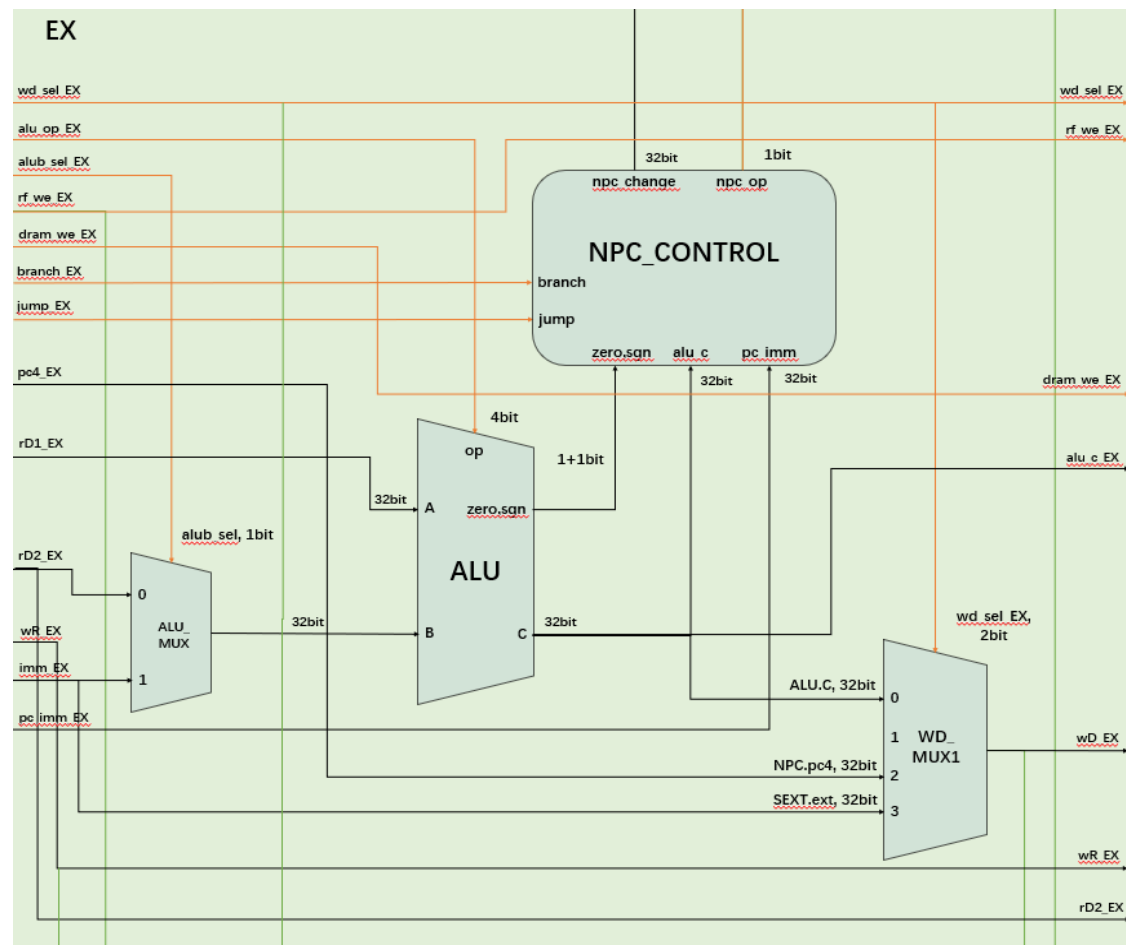
## 译指模块

**CONTROLLER:**

控制器，根据指令生成各个器件的控制信号。**和单周期数据通路的控制器不同。**由于生成 npc\_op 所需要的 zero 和 sgn 要到 EX 阶段才能得到，故转而生成判断 jal/jalr 和 B 型指令的控制信号 jump 和 branch，传到 EX 阶段，到那时再做处理。

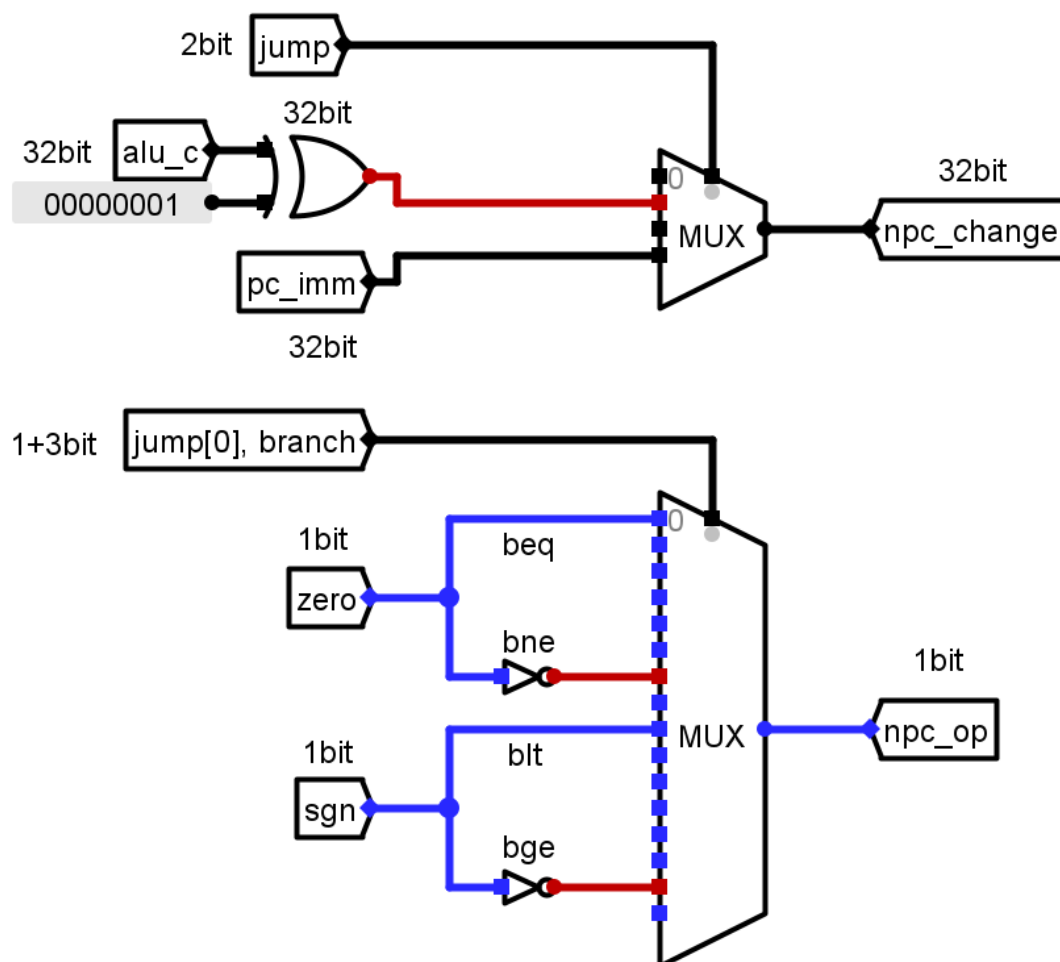
SEXT 和 RF 模块与单周期中相应模块原理相同。

## 执行模块



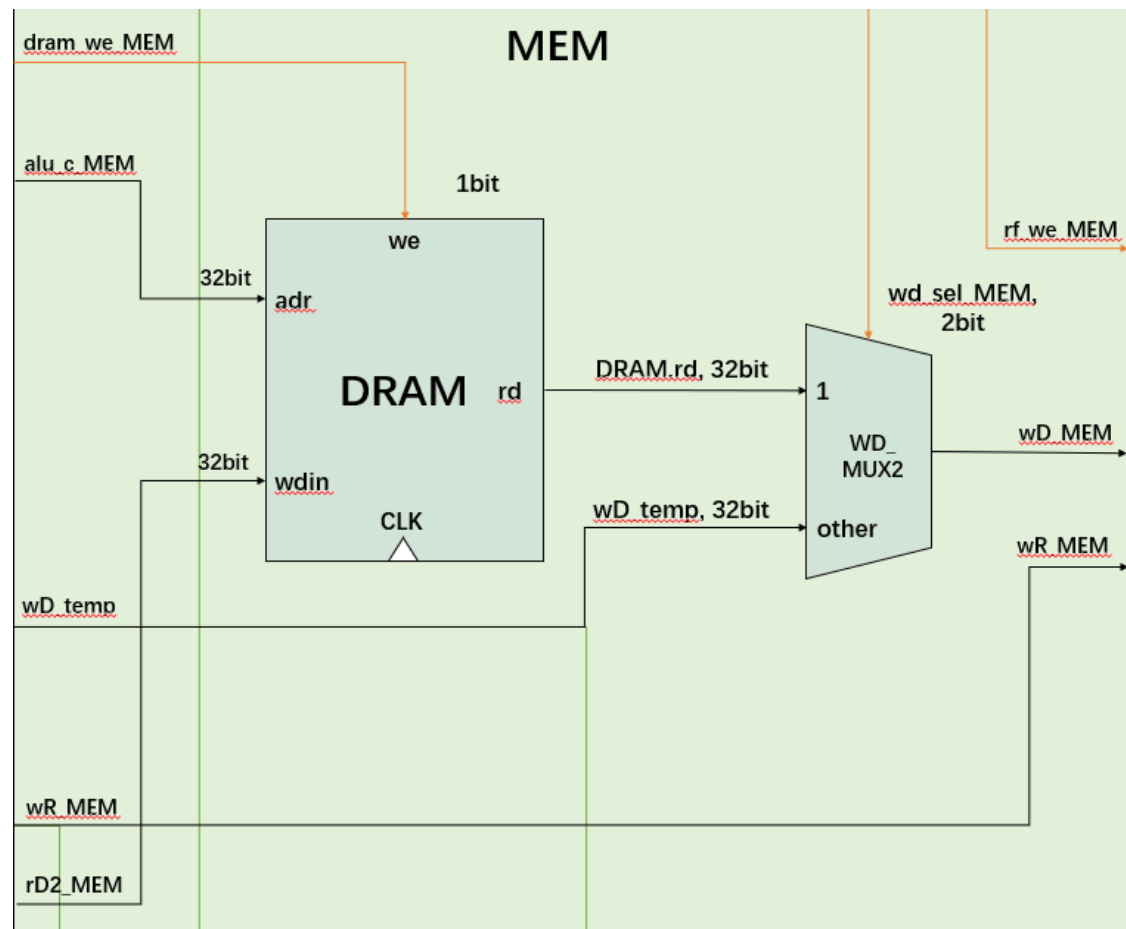
ALU\_MUX 和 ALU 模块与单周期中相应模块原理相同。

NPC\_CONTROL: 根据控制信号 jump、branch、zero、sgn 得出跳转标志 npc\_op; 根据 jump 选择 rd1 + imm 和 pc + imm 作为 npc\_change 输出。npc\_op 和 npc\_change 都将会传达给 IF 阶段的 NPC。



WD\_MUX1: 写回寄存器堆的数据 wD 的第一次选择。由于还有 DRAM.rd 没有得到, 故目前是三选一。这样的话, imm 和 pc + 4 就不需要继续传递下去了。

## 存储模块



DRAM 模块与单周期中相应模块原理相同。

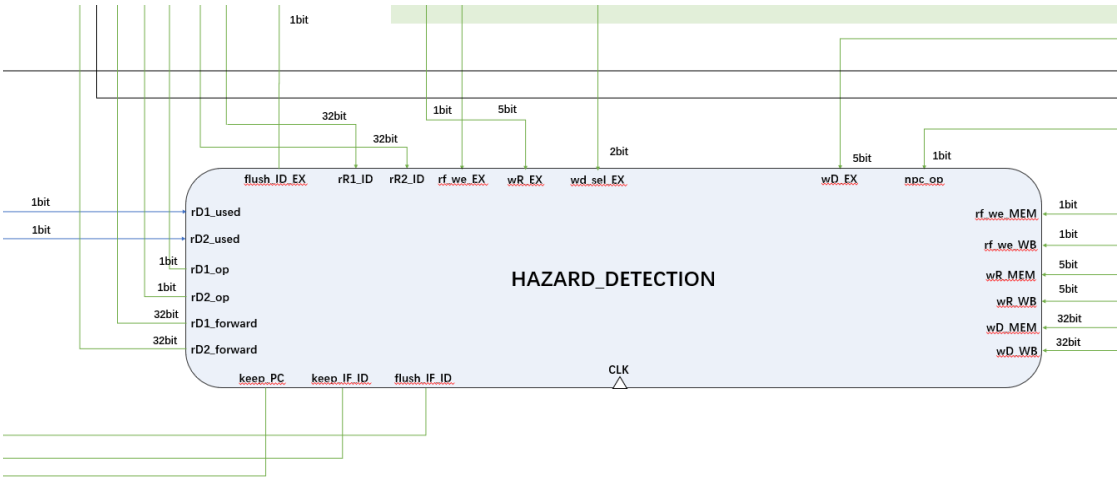
WD\_MUX2: 写回寄存器堆的数据 wD 的第二次选择。将前面得到的 wD\_temp 和 DRAM.rd 再做一次选择, 得到最后的 wD。

流水线寄存器模块

REG\_IF\_ID、REG\_ID\_EX、REG\_EX\_MEM、REG\_MEM\_WB：用于存储各阶段正在执行的指令信息。

| 流水线寄存器     | 保存的信号                                                                                      | 其他功能           |
|------------|--------------------------------------------------------------------------------------------|----------------|
| REG_IF_ID  | pc、pc4、inst                                                                                | 清空、保持          |
| REG_ID_EX  | wd_sel 、 alu_op 、 alub_sel 、 rf_we 、 dram_we、 branch、 jump、 pc_imm、 imm、 pc4、 wR、 rD1、 rD2 | 清空、接收前递控制信号和数据 |
| REG_EX_MEM | wd_sel 、 rf_we 、 dram_we 、 wR 、 wD 、 alu_c、 rD2                                            | -              |
| REG_MEM_WB | rf_we、 wR                                                                                  | -              |

冒险检测器模块



HAZARD\_DETECTION：检测并处理数据冒险与控制冒险：接收用于判断冒险的控制信号和数据，发出前递、停顿、清除的控制信号和相关数据。

## 检测冒险

### 1. 数据冒险中 RAW - A/B/C 型冒险的检测

```
// 数据冒险: RAW
// RAW - A 相邻
wire RAW_A_rD1 = (wR_EX == rR1_ID) && rf_we_EX && rD1_used && wR_EX;
wire RAW_A_rD2 = (wR_EX == rR2_ID) && rf_we_EX && rD2_used && wR_EX;
// RAW - B 间隔一条
wire RAW_B_rD1 = (wR_MEM == rR1_ID) && rf_we_MEM && rD1_used && wR_MEM;
wire RAW_B_rD2 = (wR_MEM == rR2_ID) && rf_we_MEM && rD2_used && wR_MEM;
// RAW - C 间隔两条
wire RAW_C_rD1 = (wR_WB == rR1_ID) && rf_we_WB && rD1_used && wR_WB;
wire RAW_C_rD2 = (wR_WB == rR2_ID) && rf_we_WB && rD2_used && wR_WB;
```

核心条件是  $wR\_EX/MEM/WB == rR1/rR2$ , 附加的条件用于排除其他错误检测情况, 如: 此阶段 RF 需要可写, 即  $rf\_we \neq 0$ ;  $rD1/rD2$  必须被使用;  $wR$  不可以是  $x0$ 。

### 2. 数据冒险中载入-使用型冒险的检测

```
wire load_use_hazard = (RAW_A_rD1 || RAW_A_rD2) & (wd_sel_EX == `DRAM_RD);
```

从形式上看, 载入-使用型冒险接进于 RAW-A 冒险, 两条指令相邻; 但区别是上一条指令是 LOAD 指令, 这是关键。

### 3. 控制冒险的检测

```
wire control_hazard = npc_op;
```

$npc\_op$  也即跳转的标志。只要发生跳转, 就一定有控制冒险的存在。

## 前递处理 RAW - A/B/C 型冒险

用前递解决数据冒险，需要将控制信号（使能）和数据送到 ID/EX 流水线寄存器中。

使能为 1 的话，就将寄存器的输出变更为前递的数据。具体如下：

```
// 可能接收前递 rD1/ rD2
always @ (posedge clk or negedge rst_n) begin
    if (~rst_n)      rD1_o <= 32'b0;
    else if (rD1_op) rD1_o <= rD1_forward;
    else             rD1_o <= rD1_i;
end

always @ (posedge clk or negedge rst_n) begin
    if (~rst_n)      rD2_o <= 32'b0;
    else if (rD2_op) rD2_o <= rD2_forward;
    else             rD2_o <= rD2_i;
end
```

对于这三种冒险来说，只要发生了这三种冒险，那么使能就应该为 1。

```
// 前递的使能
assign rD1_op = RAW_A_rD1 || RAW_B_rD1 || RAW_C_rD1;
assign rD2_op = RAW_A_rD2 || RAW_B_rD2 || RAW_C_rD2;
```

根据检测冒险的结果，填充前递的数据。

如果同时发生多种冒险，if - else 的结构也保证了相邻 > 间隔 1 条 > 间隔 2 条的优先级。

```
// if - else 体现了优先级：相邻 > 间隔 1 条 > 间隔 2 条
always @ (*) begin
    if (RAW_A_rD1)      rD1_forward = wD_EX;
    else if (RAW_B_rD1) rD1_forward = wD_MEM;
    else if (RAW_C_rD1) rD1_forward = wD_WB;
    else                rD1_forward = 32'b0;
end

always @ (*) begin
    if (RAW_A_rD2)      rD2_forward = wD_EX;
    else if (RAW_B_rD2) rD2_forward = wD_MEM;
    else if (RAW_C_rD2) rD2_forward = wD_WB;
    else                rD2_forward = 32'b0;
end
```



### 停顿 + 前递解决载入-使用型数据冒险

处理方式：1) 停顿，插入气泡（PC，IF/ID 不变；ID/EX 置 0）；2) 前递。

停顿的实现是向器件发送 keep 信号，当接收到信号时，将寄存器的输出在下一周期保持不变；清除（插入气泡）的实现是向器件发送 flush 信号，当接收到信号时，将寄存器的输出在下一周期被置 0。

可以被停顿的器件有 PC、REG\_IF\_ID；可以被清除的器件只有 REG\_IF\_ID、REG\_ID\_EX。这是由实际实现决定的。

```
always @ (*) begin
    if (load_use_hazard) keep_PC = 1'b1;
    else
        keep_PC = 1'b0;
end

always @ (*) begin
    if (load_use_hazard) keep_IF_ID = 1'b1;
    else
        keep_IF_ID = 1'b0;
end

always @ (*) begin
    if (load_use_hazard || control_hazard) flush_ID_EX = 1'b1;
    else
        flush_ID_EX = 1'b0;
end
```

### 静态分支预测解决控制冒险

静态分支预测，总是预测不跳转；清除后二条指令，即 flush IF/ID，ID/EX。

```
always @ (*) begin
    if (control_hazard) flush_IF_ID = 1'b1;
    else
        flush_IF_ID = 1'b0;
end

always @ (*) begin
    if (load_use_hazard || control_hazard) flush_ID_EX = 1'b1;
    else
        flush_ID_EX = 1'b0;
end
```

## 2.4 流水线 CPU 仿真及结果分析

要求：包含控制冒险和数据冒险三种情形的仿真截图，以及波形分析。

找冒险的办法：

查看 start 的波形，找冒险检测器 HAZARD\_DETECTION 的标志信号，就可以找到相应的数据冒险；

至于控制冒险，B 型和 jal、jalr 是必定涉及的，也很好找。

通过 pc\_IF, pc\_ID, pc\_EX, pc\_MEM, pc\_WB 可以看到指令在数据通路中的流动。

具体见下。

## RAW - A 型数据冒险（相邻）

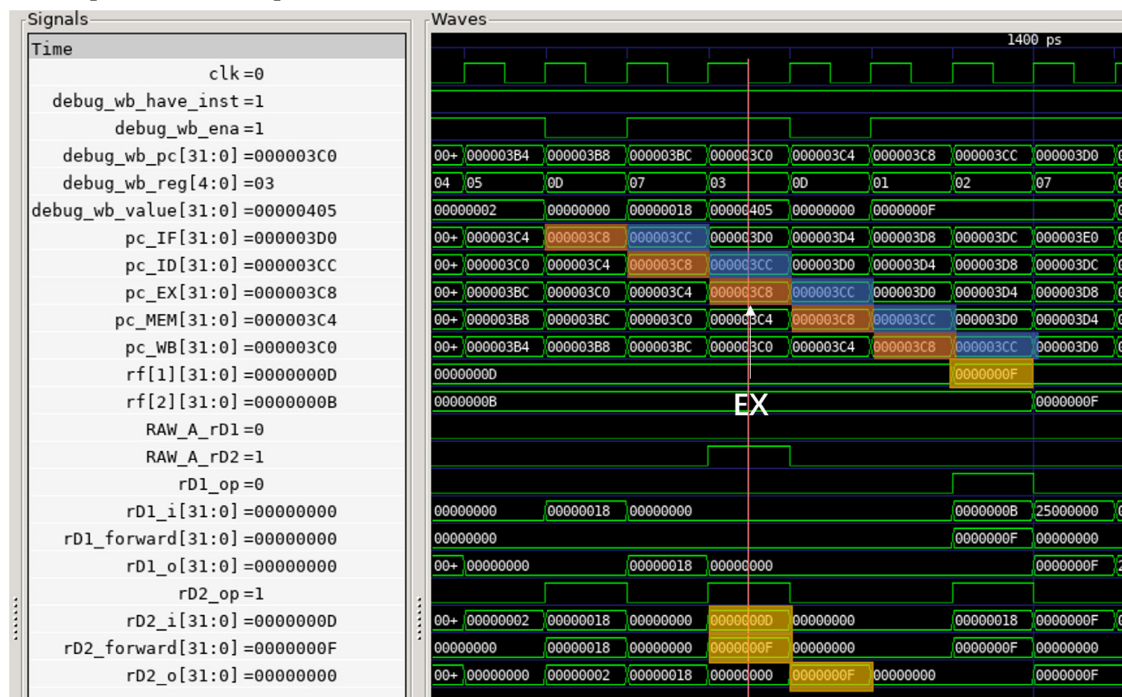
000003c8 &lt;test\_1035&gt;:

```

3c8: 00f00093      addi    x1,x0,15
3cc: 00100133      add     x2,x0,x1
3d0: 00f00393      addi    x7,x0,15
3d4: 40b00193      addi    x3,x0,1035
3d8: e8711ce3      bne     x2,x7,270 <fail>
3dc: 00140413      addi    x8,x8,1
3e0: fffff0b7      lui     x1,0xfffff
3e4: 0080a023      sw      x8,0(x1) # fffff000 <_end+0xfffffaf90>
3e8: 000f80e7      jalr    x1,0(x31)

```

关注 pc = 3c8 和 pc = 3cc, 与 x1 有关。



可以看到 x1 在 pc = 3c8 的指令的 WB 阶段结束后, 值变为 0000 000f;

而 pc = 3cc 的指令的 EX 阶段就需要使用到 x1 了, 而 x1 的值仍然是原来的 0000 000d, 于是就需要使用前递来解决问题。

注意到 pc = 3c8 的指令在 EX 阶段时, pc = 3cc 的指令在 ID 阶段;

此时, 冒险检测器检测到发生冒险, 并对 rD2 进行前递: 在 ID/EX 寄存器内, rD2\_i = 0000 000d, 这是原本的 x1 的值;

前递来的 rD2\_forward = 0000 000f, 自然需要选取前递的值作为输出 (rD2\_op = 1)。

故在 pc = 3cc 的指令到达 EX 阶段时, 从 ID/EX 寄存器得到的 rD2\_o 是 0000 000f, 成功完成了前递, 解决了数据冒险。

## RAW - B 型数据冒险（间隔一条）

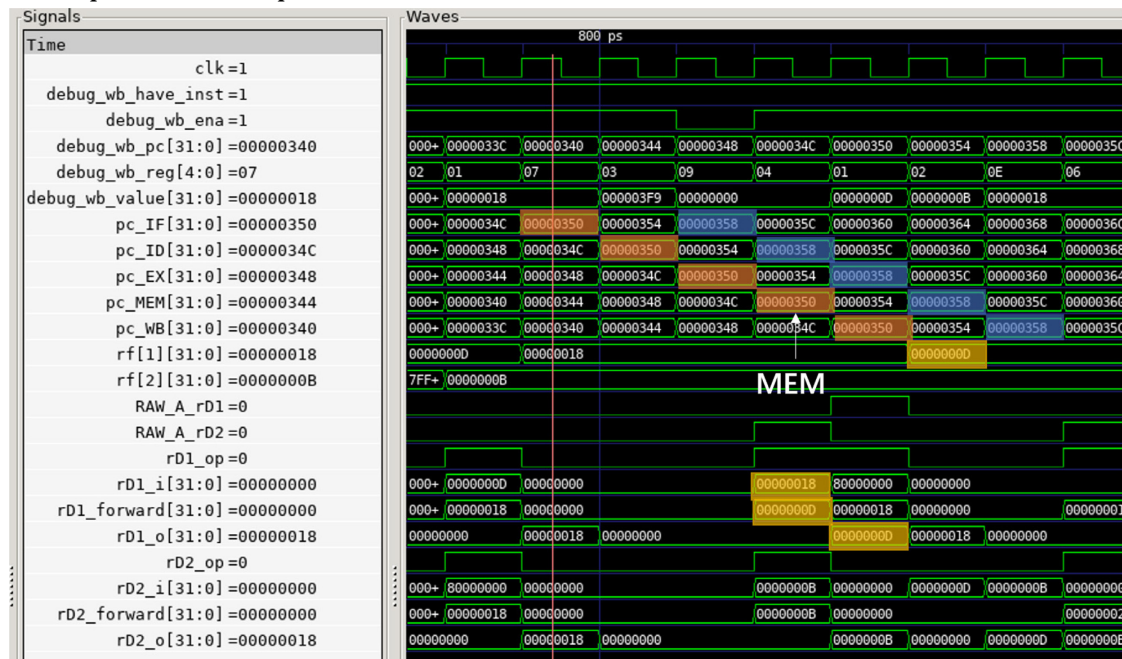
0000034c &lt;test\_1020&gt;:

```

34c: 00000213      addi    x4,x0,0
350: 00d00093      addi    x1,x0,13
354: 00b00113      addi    x2,x0,11
358: 00208733      add     x14,x1,x2
35c: 00070313      addi    x6,x14,0
360: 00120213      addi    x4,x4,1 # 1 <_start+0x1>
364: 00200293      addi    x5,x0,2
368: fe5214e3      bne     x4,x5,350 <test_1020+0x4>
36c: 01800393      addi    x7,x0,24
370: 3fc00193      addi    x3,x0,1020
374: ee731ee3      bne     x6,x7,270 <fail>

```

关注 pc = 350 和 pc = 358, 与 x1 有关。



可以看到 x1 在 pc = 350 的指令的 WB 阶段结束后, 值变为 0000 000d;

而 pc = 358 的指令的 EX 阶段就需要使用到 x1 了, 而 x1 的值仍然是原来的 0000 0018, 于是就需要使用前递来解决问题。

注意到 pc = 350 的指令在 MEM 阶段时, pc = 358 的指令在 ID 阶段;

此时, 冒险检测器检测到发生冒险, 并对 rD1 进行前递: 在 ID/EX 寄存器内, rD1\_i = 0000 0018, 这是原本的 x1 的值;

前递来的 rD1\_forward = 0000 000d, 自然需要选取前递的值作为输出 (rD1\_op = 1)。故在 pc = 358 的指令到达 EX 阶段时, 从 ID/EX 寄存器得到的 rD1\_o 是 0000 000d, 成功完成了前递, 解决了数据冒险。

## RAW - C 型数据冒险（间隔两条）

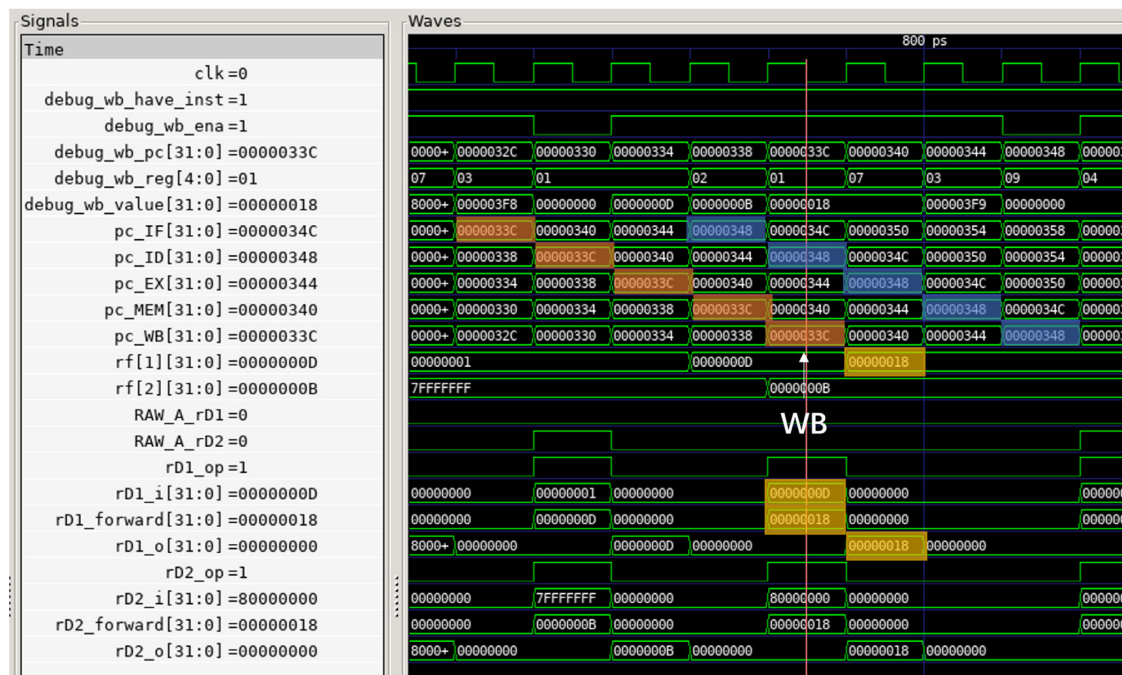
00000334 &lt;test\_1017&gt;:

```

334:  00d00093      addi    x1,x0,13
338:  00b00113      addi    x2,x0,11
33c:  002080b3      add x1,x1,x2
340:  01800393      addi    x7,x0,24
344:  3f900193      addi    x3,x0,1017
348:  f27094e3      bne x1,x7,270 <fail>

```

关注 pc = 33c 和 pc = 348，与 x1 有关。



可以看到 x1 在 pc = 33c 的指令的 WB 阶段结束后，值变为 0000 0018；

而 pc = 348 的指令的 EX 阶段就需要使用到 x1 了，而 x1 的值仍然是原来的 0000 000d，于是就需要使用前递来解决问题。

注意到 pc = 33c 的指令在 WB 阶段时，pc = 348 的指令在 ID 阶段；

此时，冒险检测器检测到发生冒险，并对 rD1 进行前递：在 ID/EX 寄存器内，rD1\_i = 0000 000d，这是原本的 x1 的值；

前递来的 rD1\_forward = 0000 0018，自然需要选取前递的值作为输出（rD1\_op = 1）。

故在 pc = 348 的指令到达 EX 阶段时，从 ID/EX 寄存器得到的 rD1\_o 是 0000 0018，成功完成了前递，解决了数据冒险。

## 载入 - 使用型数据冒险

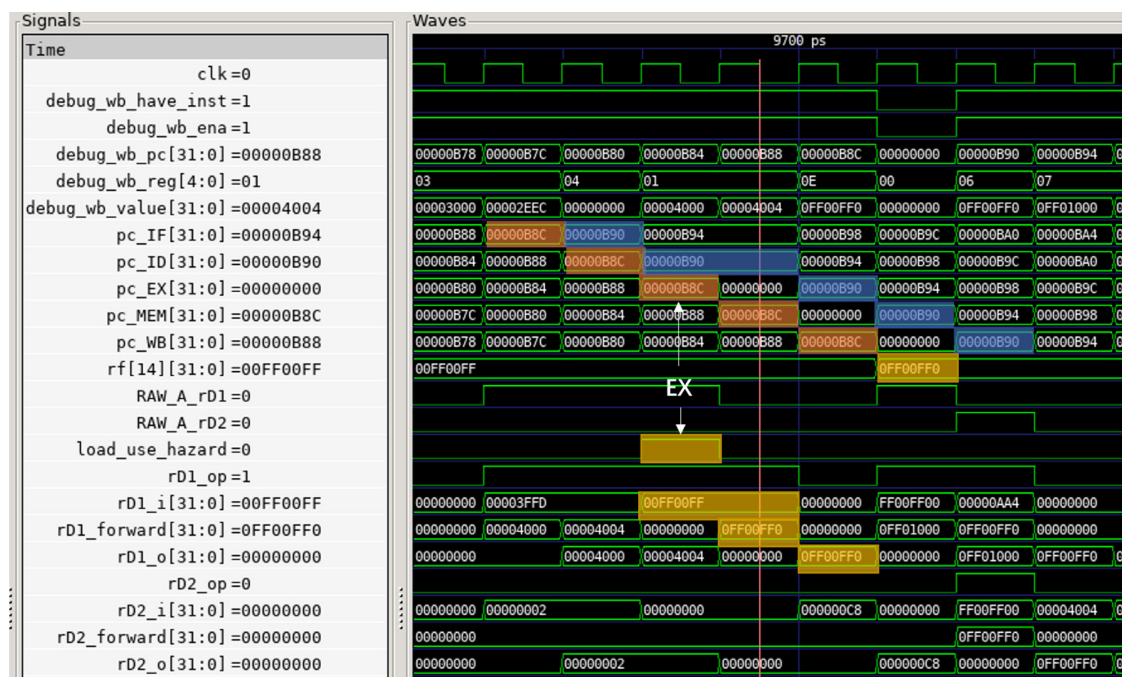
00000b78 &lt;test\_12012&gt;:

```

b78: 000031b7      lui x3,0x3
b7c: eec18193      addi x3,x3,-276 # 2eec <test_37035+0x5dc>
b80: 00000213      addi x4,x0,0
b84: 000040b7      lui x1,0x4
b88: 00408093      addi x1,x1,4 # 4004 <tdat_lw2>
b8c: 0040a703      lw x14,4(x1)
b90: 00070313      addi x6,x14,0
b94: 0ff013b7      lui x7,0xff01
b98: ff038393      addi x7,x7,-16 # ff00ff0 <_end+0xfefcf80>
b9c: ec731a63      bne x6,x7,270 <fail>
ba0: 00120213      addi x4,x4,1 # 1 <_start+0x1>
ba4: 00200293      addi x5,x0,2
ba8: fc521ee3      bne x4,x5,b84 <test_12012+0xc>

```

关注 pc = b8c 和 pc = b90, 与 x14 有关。



复习处理方式: 1) 停顿, 插入气泡 (PC, IF/ID 不变; ID/EX 置 0); 2) 前递。

首先可以看到 x14 在 pc = b8c 的指令的 WB 阶段结束后, 值变为 0ff0 0ff0, 同时这个值要等到 MEM 阶段才会从 dram 里读出;

注意到  $pc = b8c$  的指令在 EX 阶段时,  $pc = b90$  的指令在 ID 阶段;  
此时, 冒险检测器检测到发生载入 - 使用型数据冒险, 故停顿, 插入气泡:  
PC, IF/ID 不变: 可以看到  $pc\_IF = 0000\ 0B94$  保持了两个周期,  $pc\_ID = 0000\ 0b90$  也保持了两个周期;  
ID/EX 置 0:  $pc\_EX$  被归 0, 这样  $pc = b90$  的指令就会被停下一个周期。  
注意到  $pc = b8c$  的指令在 MEM 阶段时,  $pc = b90$  的指令仍在 ID 阶段, 此时已经从 dram 中读出  $0ff0\ 0ff0$  了。  
对  $rD1$  进行前递: 在 ID/EX 寄存器内,  $rD1\_i = 00ff\ 00ff$ , 这是原本的  $x14$  的值;  
前递来的  $rD1\_forward = 0ff0\ 0ff0$ , 自然需要选取前递的值作为输出 ( $rD1\_op = 1$ )。  
故在  $pc = b90$  的指令到达 EX 阶段时, 从 ID/EX 寄存器得到的  $rD1\_o$  是  $0ff0\ 0ff0$ , 成功完成了前递, 解决了载入 - 使用型数据冒险。



## 控制冒险

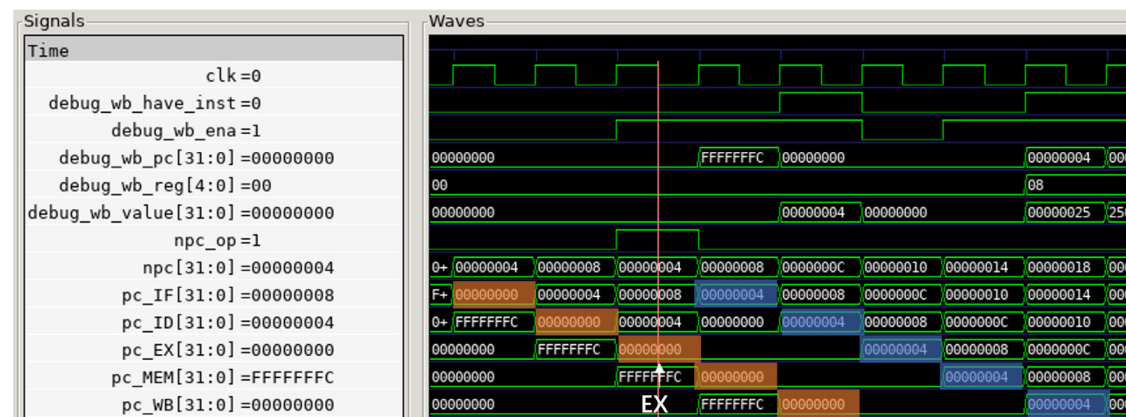
```

00000000 <_start>:
    0:  0040006f          jal x0,4 <reset_vector>

00000004 <reset_vector>:
    4:  02500413          addi   x8,x0,37
    8:  01841413          slli   x8,x8,0x18
   c:  fffff0b7          lui    x1,0xfffff
   10:  0080a023          sw    x8,0(x1) # fffff000 <_end+0xfffffaf90>
   14:  27c00fef          jal    x31,290 <n1_add_test>
   18:  00000013          addi   x0,x0,0

```

关注  $pc = 0$  的 jal 指令:



复习处理方式:静态分支预测,总是预测不跳转;清除后二条指令,即 flush IF/ID, ID/EX。

$pc = 0$  的 jal 指令,当移动到 EX 阶段,由于是 jal 指令,故控制信号 npc\_op 被赋值为 1,作为跳转的标志;

此时,冒险检测器检测到发生控制冒险,清除后二条指令,即 flush IF/ID, ID/EX,于是在下一个周期,pc\_EX, pc\_ID 就被清为 0 了。

并且, NPC 的 npc 也被修正为 4,于是在下一个周期,PC 的 pc 值就被修正为 4 了。

(PS: 在想要是  $imm = pc + 4$  的话是不是就别跳转了,就如同数据冒险中 x0 不需要被前递一样)

pc = 14 的 jal 指令也是同理。



### 3 设计过程中遇到的问题及解决方法

要求：包括设计过程中遇到的有价值的错误，或测试过程中遇到的有价值的问题。所谓有价值，指的是解决该错误或问题后，能够学到新的知识和技巧，或加深对已有知识的理解和运用。

#### 3.1 ALU 模块设计的细微差别

##### 3.1.1 如何发现

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> `include "param.v"  module ALU (     // 控制信号     input wire [3:0] op ,     // 数据信号     input wire [31:0] A ,     input wire [31:0] B ,      output wire [31:0] C ,     output wire zero,     output wire sgn );  /*  * 简单的说，在 RV-32I 中，实现移位指令时，B 有用的位数只有低五位  * 详见 RISC-V 手册 P159  */  wire [4:0] shamt = B[4:0];  reg [31:0] calc;  always @ (*) begin     case (op)         `AND : calc = A &amp; B;         `OR : calc = A   B;         `ADD : calc = A + B;         `SUB : calc = A + (~B) + 1; // A - B = A + B' + 1         `XOR : calc = A ^ B;         `SLL : calc = A &lt;&lt; shamt;         `SRL : calc = A &gt;&gt; shamt;         `SRA : calc = \$signed(A) &gt;&gt;&gt; shamt; // \$signed(A) 强调 A 有符号         default: calc = 32'b0;     endcase end  assign C = calc;  assign zero = (calc == 32'b0) ? 1'b1 : 1'b0; assign sgn = calc[31];  endmodule </pre> | <pre> `include "param.v"  module ALU (     // 控制信号     input wire [3:0] op ,     // 数据信号     input wire [31:0] A ,     input wire [31:0] B ,      output wire [31:0] C ,     output wire zero,     output wire sgn );  /*  * 简单的说，在 RV-32I 中，实现移位指令时，B 有用的位数只有低五位  * 详见 RISC-V 手册 P159  */  wire [4:0] shamt = B[4:0];  assign C = (op == `AND) ? (A &amp; B) :     (op == `OR) ? (A   B) :     (op == `ADD) ? (A + B) :     (op == `SUB) ? (A + (~B) + 1) : // A - B = A + B' + 1     (op == `XOR) ? (A ^ B) :     (op == `SLL) ? (A &lt;&lt; shamt) :     (op == `SRL) ? (A &gt;&gt; shamt) :     (op == `SRA) ? (\$signed(A) &gt;&gt;&gt; shamt) : // \$signed(A) 强调 A 有符号     32'b0;  assign zero = (C == 32'b0) ? 1'b1 : 1'b0; assign sgn = C[31];  endmodule </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

（左为修改前，右为修改后）

在实现单周期上板之后，为了测试最高频率，需要将代码优化，故而将大多数的 reg 类型的变量修正为了 wire 类型，同时将大量的 always 块修改为 assign 和三目运算符。

本来预期可以达到更高的频率，但是令人遗憾的是，经过这样的修改后，连单周期的最低要求 25MHz 也达不到，上板 trace 的结果只显示到 2500 0010，并没有到预期的 2500 0018。

针对修改前后的差别经行漫长的上板排查后，终于定位到 ALU 上，只需要把 ALU 修改为原来的样式就可以跑到 2500 0018；此外，将 assign 和三目的组合去运行在线 trace 也无法通过所有测试，没有通过的指令是 sra 和 srai。

```

===== Testing bgeu =====
Peripheral name: MONITOR      base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit      base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
[difftest] Test Failed!
===== Diffrence =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x0000001c      0x00000014
debug_wb_ena      0              0
debug_wb_reg      1674683152      4
debug_wb_value    0x00000001      0xfffffffffe
make: *** [Makefile:16: run_for_python] Error 255
===== bgeu END =====

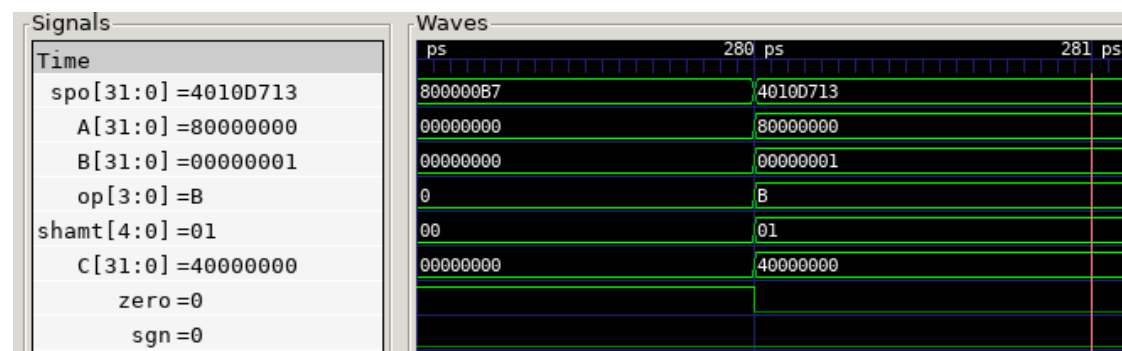
===== SUMMARY =====
Passed Tests:
and, addi, add, srl, simple, sw, sll, xori, beq, lw, ori, bne, bge, jal, or, jalr, srli, andi, slli, sub, blt, xor
Failed Tests:
lhu, sb, lh, sra, lui, slt, slti, start, srai, auipc, sltiu, lb, bltu, sltu, sh, lbu, bgeu
200111407@comp2:~/cdp-tests$ make run TEST=start
make[1]: Entering directory '/home/students/200111407/cdp-tests/obj_dir'
make[1]: Nothing to be done for 'default'.
make[1]: Leaving directory '/home/students/200111407/cdp-tests/obj_dir'
Peripheral name: MONITOR      base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit      base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
Digit: 0x25000000
Digit: 0x25000001
Digit: 0x25000002
Digit: 0x25000003
Digit: 0x25000004
Digit: 0x25000005
Digit: 0x25000006
Digit: 0x25000007
Digit: 0x25000008
Digit: 0x25000009
Digit: 0x2500000a
Digit: 0x2500000b
Digit: 0x2500000c
Digit: 0x2500000d
Digit: 0x2500000e
Digit: 0x2500000f
Digit: 0x25000010
[difftest] Test Failed!
===== Diffrence =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x000010ac      0x000010ac
debug_wb_ena      1              1
debug_wb_reg      1              1
debug_wb_value    0xff000000      0x01000000
make: *** [Makefile:13: run] Error 255
200111407@comp2:~/cdp-tests$

```

运行 `srai` 的具体测试，通过查看反汇编和波形，发现运算出错的具体情况，本来应该进行的算术右移变成了逻辑右移：`A = 8000 0000`，`B = 01`，即 `8000 0000` 算术右移 1 位，结果本应该是 `c000 0000`，但是结果却是 `4000 0000`。

```
200111407@comp2:~/cdp-tests$ make run TEST=srai
make[1]: Entering directory '/home/students/200111407/cdp-tests/obj_dir'
make[1]: Nothing to be done for 'default'.
make[1]: Leaving directory '/home/students/200111407/cdp-tests/obj_dir'
Peripheral name: MONITOR      base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
[difftest] Test Failed!
===== Difference =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x0000001c      0x0000001c
debug_wb_ena      1                1
debug_wb_reg      14              14
debug_wb_value    0xc0000000      0x40000000
make: *** [Makefile:13: run] Error 255
```

```
00000018 <test_3>:
18: 800000b7      lui x1,0x80000
1c: 4010d713      srai x14,x1,0x1
20: c00003b7      lui x7,0xc0000
24: 00300193      addi x3,x0,3
28: 28771a63      bne x14,x7,2bc <fail>
```



即使是查看波形，也只能得出计算出错的结论，并不能带来更多的成果（毕竟算错了就是算错了）。

即使是调整 `SRA 在三目中的位置也无济于事。

已经知道了导致错误的表层原因，却不知道深层的原因。

### 3.1.2 理论解释

在老师的指点下，意识到三目运算是一种变种的 if-else 语句，以及理解了 case 语句和 if-else 语句的差别。

|  |       |  |          |  |         |  |
|--|-------|--|----------|--|---------|--|
|  |       |  | case     |  | if-else |  |
|  | 逻辑上   |  | 各分支间无优先级 |  | 有优先级    |  |
|  | 运行时间上 |  | 并行判断     |  | 串行判断    |  |

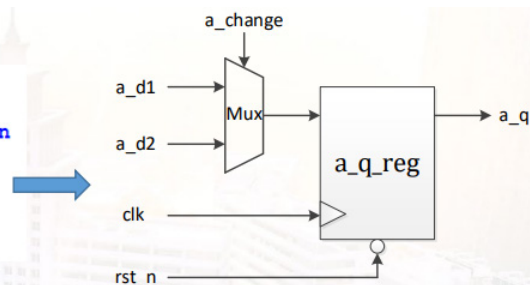
那么在判断到 `op == `SRA` 时，在时序上就可能出现问题，导致错误；

或者说，在实现的时候，if-else 的电路实现是很长的一条通路，这样是不好的。

另外，简单的把 reg 改为 wire 并不会起到实际的作用，因为不和时序逻辑相关的 reg，在综合时还是会被综合为连线。

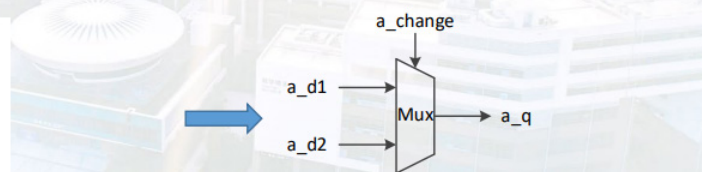
#### ◆ 时序逻辑

```
reg [3:0] a_q;
always @ (posedge clk or negedge rst_n) begin
    if (~rst_n)
        a_q <= 4'h0;
    else if (a_change)
        a_q <= a_d1;
    else
        a_q <= a_d2;
end
```



#### ◆ 组合逻辑

```
reg [3:0] a_q;
always @ (*) begin
    if (a_change)
        a_q = a_d1;
    else
        a_q = a_d2;
end
```



所以之前的优化都是无效的。



## SoC 芯片设计——为什么使用 assign 语句，来避免使用 if-else 或者 case 来设计

## 电路。摆渡沧桑的博客-CSDN 博客用 assign 语句设计四选一选择电路

这个参考我不好说，他声称可以有办法在使用三目的同时达成并行。但是用了他的办法却没啥效果，trace 过不了。

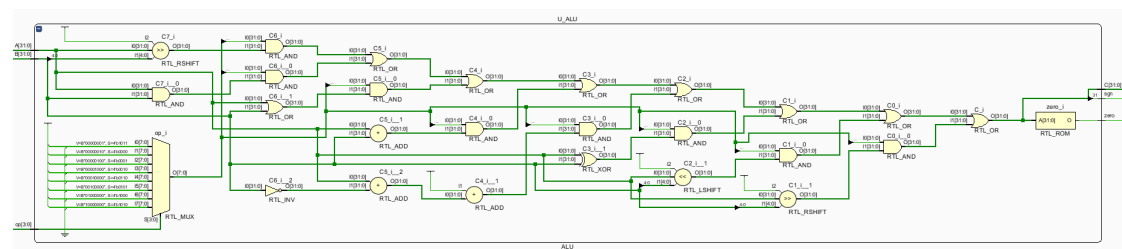
```

wire op_sra = (op == `SRA);
wire op_and = (op == `AND);
wire op_or  = (op == `OR);
wire op_add = (op == `ADD);
wire op_sub = (op == `SUB);
wire op_xor = (op == `XOR);
wire op_sll = (op == `SLL);
wire op_srl = (op == `SRL);

assign C = (
    ({32{op_sra}} & ($signed(A) >>> shamt) ) |
    ({32{op_and}} & (A & B) ) |
    ({32{op_or}} & (A | B) ) |
    ({32{op_add}} & (A + B) ) |
    ({32{op_sub}} & (A + (~B) + 1) ) |
    ({32{op_xor}} & (A ^ B) ) |
    ({32{op_sll}} & (A << shamt) ) |
    ({32{op_srl}} & (A >> shamt) )
);

```

查看实际 RTL 分析结果，也没有避免 if-else 原本的问题。



## 3.2 检测数据冒险的判断条件

### 3.2.1 如何发现

```
// 数据冒险: RAW
// RAW - A 相邻
wire RAW_A_rD1 = (wR_EX == rR1_ID) && rf_we_EX && rD1_used && wR_EX;
wire RAW_A_rD2 = (wR_EX == rR2_ID) && rf_we_EX && rD2_used && wR_EX;
// RAW - B 间隔一条
wire RAW_B_rD1 = (wR_MEM == rR1_ID) && rf_we_MEM && rD1_used && wR_MEM;
wire RAW_B_rD2 = (wR_MEM == rR2_ID) && rf_we_MEM && rD2_used && wR_MEM;
// RAW - C 间隔两条
wire RAW_C_rD1 = (wR_WB == rR1_ID) && rf_we_WB && rD1_used && wR_WB;
wire RAW_C_rD2 = (wR_WB == rR2_ID) && rf_we_WB && rD2_used && wR_WB;
```

指导书上，关于 RAW 冒险的三种情况的判断条件，只有前三个被提及，分别是寄存器（号）和 wR 有对应关系，RF 可写，以及寄存器确实被使用。但是仅凭借这三个条件，并不能成功的通过 trace。

联系到计算机组成原理课上所学，尝试为其添加了 wR 不可为 0 的判断条件，也就是如果相关联的寄存器是 x0 的话，就不构成数据冒险；加上该条件后就可以通过 trace。

如果相关联的寄存器是 x0，确实不构成数据冒险的条件；但是为什么不加上这个判断条件就无法通过呢？

### 3.2.2 理论解释

浅显的考虑一下，即使是前递也是可以前递回 0 的吧？但是实际的情况并不是这样。

RF 可以保证若 wR 是 0，wD 不管为何值都只能写回 0，即 wD 本身不一定是 0，例如：（假设  $x1 = 1$ ,  $x3 = 3$ ）

```
sub x0, x1, x3
```

写回的 wR 是 -2，但是被 RF 中检测出来并强制写回 0 了。但是在前递的时候是没有这样的判断的，因此需要加上某种判断，可以从一开始就判断不构成数据冒险，也可以是在后面判断 wR 为 0 则不前递，或者前递 0，都是可以的，但显然放在最开始是最合适的。

### 3.2.3 实践理论

不加上 wR 不可为 0 的判断条件，测试 add 指令。

```
200111407@comp2:~/cdp-tests$ make run TEST=add
make[1]: Entering directory '/home/students/200111407/cdp-tests/obj_dir'
make[1]: Nothing to be done for 'default'.
make[1]: Leaving directory '/home/students/200111407/cdp-tests/obj_dir'
Peripheral name: MONITOR          base: 0x80000000      addr len: 0x00000008
Peripheral name: Digit base: 0xfffff000      addr len: 0x00000004
[mycpu] Resetting ...
[INFO] Data RAM initialized with meminit.bin
[INFO] Instruction ROM initialized with meminit.bin
[mycpu] Reset done.
[difftest] Test Start!
[difftest] Test Failed!
===== Difference =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x000004d4      0x000004d4
debug_wb_ena      1                1
debug_wb_reg      7                7
debug_wb_value    0x00000000      0x0000002e
make: *** [Makefile:13: run] Error 255
```

查看反汇编：

```
000004c8 <test_38>:
4c8: 01000093      addi x1,x0,16
4cc: 01e00113      addi x2,x0,30
4d0: 00208033      add x0,x1,x2
4d4: 00000393      addi x7,x0,0
4d8: 02600193      addi x3,x0,38
4dc: 00701463      bne x0,x7,4e4 <fail>
4e0: 00301e63      bne x0,x3,4fc <pass>
```

可知  $x1 = 16$ ,  $x2 = 30$ ;  $pc = 4d0$  的指令  $x0$  被写回的结果  $wD = 46$ , 即  $2e(16$  进制), 那么  $x7$  就会接收到这个前递的值, 从而计算错误, 故而得出错误的结果  $2e$ 。

和上面的解释匹配的很好。



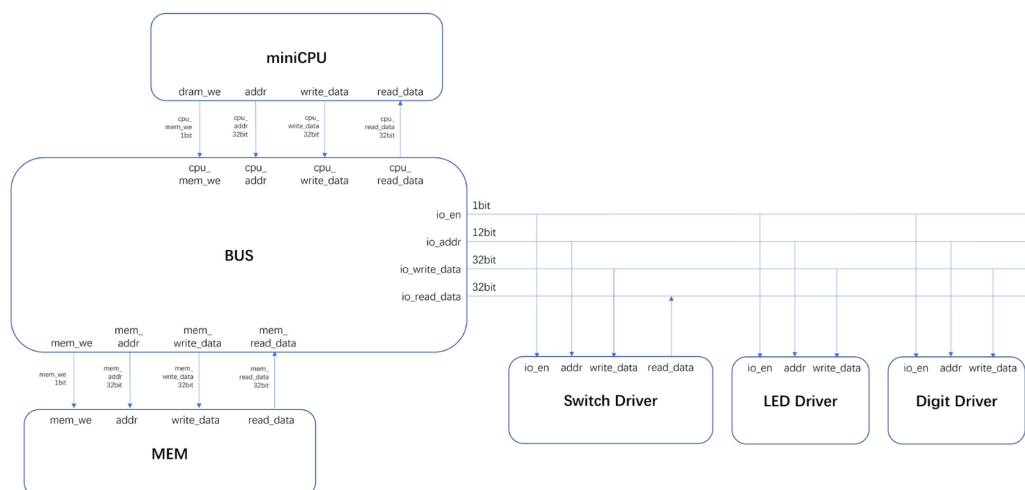
## 4 总结

要求：谈谈学完本课程后的个人收获以及对本课程的建议和意见。请在认真总结和思考后填写总结。

通过学习这门课程，让我对计算机组成原理课内所学的知识有了进一步的理解和运用，这是仅学习理论知识所无法得到的宝贵经验。

大体知识都已经在计算机组成原理中学习，真的到实践的时候细节会更多、更复杂，但也可以反过来去深刻的理解某些理论，使得学过的理论有了实践支撑。我想这就是“实践是理论的基础，即实践对理论具有决定作用；理论对实践有反作用，科学的理论对实践具有积极的指导作用”的道理吧。

让我感触比较深的有总线外设的部分。模仿实验一的 SoC，设计了类似的总线外设



其实在我学习总线这一章节的时候，并没有对总线有很清除的理解，虽然知道基本概念、分类、结构的基础知识，但是对总线的具体实现完全不理解，比如不明白为什么地址总线是指向 I/O 设备的，以这个问题为代表的诸多问题表现了我对总线的不理解，也让我在设计结构的时候一筹莫展，直到去看了实验一的 SoC，我才明白应该如何设计，并且在实现完成之后，我对基本的总线外设有了新的理解：

以 CPU 的视角来看，CPU 的 `dram_we` 是驱使外部器件读写的动力，`addr` 是区别外部设备的方法，`read_data` 和 `write_data` 是读写的数据。

之前没有总线外设的时候，CPU 是可以直接驱使 DRAM 读写的；在有了外设之后，为了区分 DRAM 和众多外设，才需要 BUS 来进行处理，此时的 `dram_we` 就不是原来的 DRAM 的读写使能了，而是和外部器件读写的请求，这个外部器件可以是 DRAM，也可以是外设；这个读写请求会和传来的地址一起，用于生成外部器件的读写使能（笼统说法），比如传来的是外设的地址，那么外设的使能

(注意是使能不是读写使能)为 1, 而 DRAM 的使能为 0, 然后再用 `dram_we` 来决定读写使能。

至于怎么区分 DRAM 和外设, 外设之间怎么区分, 就是地址的作用。我去规定哪些地址是 DRAM 用的, 哪些地址又是哪些外设用的。

那么对于外设而言, `addr` 数据线毫无疑问的是指向外设的, 因为 `addr` 的源头是 CPU; 但是外设内部是可以对连接来的 `addr` 进行判断的, 结合使能 `io_en`, 就能判断是否是选中了自己。

当然现在设计的这个外设在上略微的简单了点, 还是很基础啦, 距离功能较为完善的总线外设还有很远。

最后, 虽然计组学的不是很好, 但是我觉得这不妨碍我认真学这门实践课。全身心的投入到这门课里是一种很好的体验 (对比: 之前软件构造实践时间非常紧, 设计也不完善, 只做个半成品其实是很让人气馁的); 而且能够利用所学知识造一个简单的 CPU, 难道这不酷吗?!

总之, 学习这门课我收获了很多, 也很感谢老师和同学的教导!